Build Responsive Real-World Websites with HTML and CSS

Section 1: Welcome and First Steps

1. HTML & CSS to build beautiful and responsive and real-world website
2. Topics:
    1. Web design
    2. HTML
    3. CSS
    4. Responsive Design

Course Structure and Projects

```
1. 9 sections
2. 35 hours
3. Sections:
    1. Welcome and first steps
        1. High level overview of web development
        2. Simple web-page
    2. HTML Fundamentals
        1. Small project for solid foundation
    3. CSS Fundaments
        1. Foundation
            1. Styling text
            2. CSS box-model
            3. Sizing & positioning elements
            4. Reading documentation
            5. Debugging
    4. Building Layouts
        1. Using CSS
        2. How to arrange elements on a page in a logical way
            1. Multiple ways
                1. Flex box
                2. CSS grid
    5. Web Design Framework
        1. Special section
        2. Easy to use design rules & design guidelines
            1. To make them beautiful and professional looking
    6. Components and layouts
        1. Common website components
        2. Layout patterns
            1. Examples
        3. Building some components with HTML and CSS
    7. Omnifood: Desktop
        1. How to plan and sketch a website
    8. Omnifood: Responsive
        1. How to make it responsive
    9. Omnifood: Optimizations
        1. How to handle an entire project
```

Read Before You Start!

1. Download start code:
    1. Starter and final code and FAQs on GitHub
2. Download course material:
    1. Additional materials from lecture:
        1. **theory-lectures-v2-BEST.pdf**
            1. Slides for all theory lectures with GOOD image quality
        2. **theory-lectures-v2-SMALLER.pdf**
            1. Slides for all theory lectures with AVERAGE image quality (158 MB download)
        3. **all-design-guidelines.pdf**
            1. Summary of all the web design rules and guidelines we will study in Section 5
3. Community & Resources
    1. Student community
        1. by clicking here!
    2. online tools
4. Don't use lecture numbers when taking nodes
    1. They will change each time something is updated

A High-Level Overview of Web Development

1. Topics:
    1. Clients and servers
    2. Frontend and backend development
    3. Static and dynamic websites
    4. Core languages and core technologies
2. Front-end vs Back-end development
    1. Website: http://omnifood.dev
        1. Browser will send a request to the web server (hosted on the internet)
            1. Server - a computer that is connected to the internet and able to receive requests
        2. When the server receives the request, it will take all the files that will make up the response and send them back to the browser
            1. Server sends response to the browser
            2. Files:
                1. index.html
                2. style.css
                3. script.js
                4. image.jpg
            3. Browsers can understand html, css, and js
                1. We need to write websites in html, css, and js
            4. Browser takes the code an renders a website

3. Front-end web development
    1. Developers writing code to display in browser
    2. Static web-site:
        1. Files are simply stored and sent to browser
            1. There are no transformations
    4. Back-end development
        1. A request is sent to a server
        2. A lot of content is changing all the time
            1. Example: New courses, new ratings, etc...
            2. This needs an application running on server and a big database (containing content displayed on the website)
        3. Backend languages are used:
            1. Node.js
            2. PHP
            3. Python
        4. The code takes data out of database and assembles the data into final files that will be sent as a response
        5. Dynamic website: The website is dynamically assembled from different pieces on the server
            1. Happens each time someone visits the website
                1. A new version of website is generated
        6. The files are sent to browser and a website is rendered
3. The 3 languages of the front-end
    1. HTML (Nouns - <p> means "paragraph")
        1. Responsible for the content of the page
            1. Text
            2. Images
            3. Buttons
            4. ...
    2. CSS (Adjectives - p { color: red } means "the paragraph text is red")
        1. Responsible for presentation
            1. Styling
            2. Laying out elements
    3. JS (Verbs - p.hide(); means "hide the paragraph")
        1. Programming language of the front-end
        2. It allows us to add dynamic and interactive effects to web-pages
        3. It allows us to:
            1. Manipulate the content or the css
            2. Load data from web-servers
            3. Build the entire front-end applications (web-applications)

Setting Up Our Code Editor

1. Code editor - a special software that allows us to write programming code (HTML, CSS) in an easy way
2. VS Code - best editor for web-development
    1. Download and install
    2. Installing extensions:
        1. prettier
            1. Settings
                1. Search: default formatter
                    1. Prettier - Code formatter
                2. Search: format on save
                    1. Check
                3. Search: auto save
                    1. onFocusChange
                        1. When we go to another tab or leave the window
                4. Search: Tab Size
                    1. 2
        2. one monokai
            1. Shows colors
            2. Settings
                1. Color Theme
                    1. Select a theme
    3. Settings
        1. File Icon theme
            1. Seti

Your Very First Webpage!

1. VS Code
    1. Explorer tab
        1. New folder: 01-Test
        2. Open Folder
            1. 01-Test
        3. New File
            1. index.html - default name of the first page of any website - entry point to any website

                1. ! tab

                ```
                <title>My webpage</title>
                ...
                <body>
                    <h1>Hello World!</h1>
                    <p>My name is Abdullah, and ths is my webpage</p>
                </body>
                ```

                2. Open it in Google Chrome (use this browser to test)

Downloading Course Material

1. Starter files
    1. https://github.com/jonasschmedtmann/html-css-course
        1. starter
            1. 02-HTML-Fundamentals
                1. Images
                2. Documents
                3. ...
        2. FAQ:
            1. Read through the questions
        3. Code > Download Zip

Watch Before You Start!

1. Considerations:
    1. Don't get overwhelmed
        1. Go through the sections even if we don't understand but ultimately we will understand
    2. Code along
    3. Take notes
        1. Syntax
        2. Theory
        3. (everything)
    4. Try coding challenges
        1. Must try
            1. Re-watch if stuck
    5. Go in sequence (don't skip sections)
        1. Review code
        2. Review projects
        3. Review notes
        4. Write own code (before moving on)
    6. Try to solve errors by ourselves
        1. Developer must fix bugs
            1. Check if someone had the same problem
            2. Codepend.io and send link
    7. The code works on every platform
    8. Have fun

Section 2: HTML Fundamentals

Section Intro

1. Fundamental language: HTML
    1. Example project
        1. Learn important example elements

Introduction to HTML

1. What is HTML?
    1. HyperText Markup Language
        1. It is a core web-technology
    2. HTML is a **markup language** that web developers can use to structure and describe the content of a webpage (not a programming language)
    3. HTML consists of **elements** that describe different types of content:
        1. Paragraph
        2. Links
        3. Headings
        4. Images
        5. Videos
        6. ...
    4. Web browsers understand HTML and **render HTML code as websites**
2. Anatomy of an HTML element:

```
<p>HTML is a markup language</p>
```

    1. Three parts:
        1. Opening tag: Name of the element wrapped in < and >
        2. Content: Content of the element, in this example text.
            1. But it might be another element (child element)
            2. Some elements have no content (e.g.)
        3. Closing tag: Same as opening tag, but with a `/`.
            1. When element has no content, it's omitted

HTML Document Structure

1. Project: The code magazine
    1. Blog post - to learn the fundamentals of HTML
2. Structure:
    1. Open downloaded folder
        1. copy **starter / 02-HTML-Fundamentals** to a different directory
    2. Open folder in VS Code
        1. New file: index.html

```
<!DOCTYPE html> <!-- Tells the browser that this document uses HTML -->
<html>
    <head>
        <title>The Basic Language of the Web: HTML</title>
    </head>
    <body>
        <h1>The Basic Language of the Web: HTML</h1>
    </body>
</html>
```

1. Turning off auto-close
    1. Settings
        1. Search: Auto Closing Tags
            1. Turn it off
2. Each HTML document must start with `html` tag
3. `<head>` - for things not visible in the browser window
    1. Title
    2. Additional info about the page
    3. Links to CSS files
    4. ...
4. Open the file in browser
5. `<!DOCTYPE html>` - tells a browser to use HTML 5 specification to render the page

Text Elements

1. Headings: Used to break up big block of text into logical sections
    1. It is used to add title to each of the sections
    2. Hierarchy of headings for hierarchy of text.
    3. Code:

```
<h1>The Basic Language of the Web: HTML</h1>
<h2>The Basic Language of the Web: HTML</h2>
<h3>The Basic Language of the Web: HTML</h3>
<h4>The Basic Language of the Web: HTML</h4>
<h5>The Basic Language of the Web: HTML</h5>
<h6>The Basic Language of the Web: HTML</h6>
```

        1. Reload the browser
        2. Markup for the given structure: Copy paste the text in txt file

```
<h1><icon> The Code Magazine</h2>
<h2>The Basic Language of the Web: HTML</h2>
<h3>What is HTML?</h3>
```

    4. **Each page must have only one `h1` heading** - only one primary heading
2. Paragraph: Used to markup bigger blocks of text
    1. Copy paste the paragraphs

```
<p>...</p>
<p>...</p>
```

3. Comments: Is a way of writing something that is not visible in browser

```
<!-- comment -->
```

4. How to markup text
5. Headings:
    1. Used to break up big blocks of text into more logical sections and add a title to each of the sections
    2. There are 6 headings (hierarchy of headings)
    3. Example:

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>

<body>
  <h1>The Basic Langauge of the Web: HTML</h1>
  <h2>The Basic Langauge of the Web: HTML</h2>
  <h3>The Basic Langauge of the Web: HTML</h3>
  <h4>The Basic Langauge of the Web: HTML</h4>
  <h5>The Basic Langauge of the Web: HTML</h5>
  <h6>The Basic Langauge of the Web: HTML</h6>
</body>

</html>
```

        1. Title: h1
        2. Article Name: h2
        3. Section Name: h3
    4. Copy paste the text:

5. `<p>` - paragraph
    1. It is used to markup bigger blocks of text
6. Each page must have only one `h1` heading element.
    1. Other headings can be multiple
7. Bold:
    1. `<b>` - not a good practice
        1. Older HTML element
        2. It does not have a semantic meaning
    2. `<strong>` **(M)** - starting from HTML5
        1. It has a semantic meaning (?)
            1. Without any specific meaning
        2. It means it is an important element to stand out on the page
8. Italic:
    1. `<i>` - older version
    2. `<em>` **(M)** - emphasize
        1. It has semantic meaning
9. Example:

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>

<body>
  <!-- <h1>The Basic Langauge of the Web: HTML</h1>
  <h2>The Basic Langauge of the Web: HTML</h2>
  <h3>The Basic Langauge of the Web: HTML</h3>
  <h4>The Basic Langauge of the Web: HTML</h4>
  <h5>The Basic Langauge of the Web: HTML</h5>
  <h6>The Basic Langauge of the Web: HTML</h6> -->

  <h1>📘 The Code Magazine</h1>
  <p>Posted by <strong>Laura Jones</strong> on Monday, June 21st 2027</p>

  <h2>The Basic Language of the Web: HTML</h2>
  <p>All modern websites and web applications are built using three <em>fundamental</em> technologies: HTML, CSS and
    JavaScript.
    These are the languages of the web.</p>

  <p>In this post, let's focus on HTML. We will learn what HTML is all about, and why you too should learn it.</p>

  <h3>What is HTML?</h3>
  <p>HTML stands for <strong>H</strong>yper<strong>T</strong>ext <strong>M</strong>arkup <strong>L</strong>anguage. It's
    a markup
    language that web
    developers use to structure and describe
    the content of a webpage (not a programming language).</p>
  <p>HTML consists of elements that describe different types of content: paragraphs, links, headings, images, video,
    etc. Web browsers understand HTML and render HTML code as websites.</p>
  <h3>Why should you learn HTML?</h3>
</body>

</html>
```

More Text Elements: Lists

1. Lists
    1. Ordered Lists - numbered list
        1. `<ol>` - ordered list
            1. `<li>` - list item
2. Container of an element is the parent
    1. Elements inside a parent are the child elements
3. Line-breaks in code are ignored

Images and Attributes

1. `post-img.jpg`
2. `<img />` - special element
    1. Doesn't have content
3. Attributes: Pieces of data used to describe elements
4. `src` - used to specify image source
5. `alt` **(M)** - used to specify alternative text
    1. Describes what the image is about
        1. Why?
            1. It allows search engines to know what is in the image (otherwise they cannot know)
            2. To allow blind people to use the website
                1. Screen reader reads the alternative text
            3. If image is not found, HTML shows the description
6. `width` **(M)**
    1. Maintains aspect ratio of image
7. `<html lang="en">` **(M)**
8. `<meta>` - metadata

1. Data about data
2. Example:

```
<meta charset="UTF-8"> <!-- **(M)** -->
```

Hyperlinks

1. Links enables internet to be a world-wide-web
    1. Without links, there would be no web
2. Links that point to other pages in our own website
3. Links that point to outside our website
4. https://developer.mozilla.org/en-US/docs/Web/HTML
    1. Website to learn everything about HTML, CSS, and JavaScript
    2. URL - every website on the internet has a URL
5. Syntax:

```
<a href="<url>" target="<target>"><link-text></a>
```

    1. `<target>` - `_blank` **(M)**
        1. `_self` - same browser context
        2. `_top` - topmost ancestor of browser context or `_self` if no ancestor
        3. `_parent` - ancestor of browser context
        4. `_blank` - new browser context
6. Example:

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>Document</title>
</head>

<body>
  <!-- <h1>The Basic Langauge of the Web: HTML</h1>
  <h2>The Basic Langauge of the Web: HTML</h2>
  <h3>The Basic Langauge of the Web: HTML</h3>
  <h4>The Basic Langauge of the Web: HTML</h4>
  <h5>The Basic Langauge of the Web: HTML</h5>
  <h6>The Basic Langauge of the Web: HTML</h6> -->

  <h1>📓 The Code Magazine</h1>

  <a href="blog.html">Blog</a>
  <a>Challenges</a> <!-- it is not a link anymore, but a plain text -->
  <a href="#">Challenges</a> <!-- goes to the top of the current page-->
  <a href="#">Flexbox</a>
  <a href="#">CSS Grid</a>

  <p>Posted by <strong>Laura Jones</strong> on Monday, June 21st 2027</p>

  <img src="./images/post-img.jpg" alt="HTML code on a screen" width="500px" />

  <h2>The Basic Language of the Web: HTML</h2>

  <img src="./images/laura-jones.jpg" alt="Headshot of Laura Jones" width="50px" height="50px" />

  <p>All modern websites and web applications are built using three <em>fundamental</em> technologies: HTML, CSS and
    JavaScript.
    These are the languages of the web.</p>

  <p>In this post, let's focus on HTML. We will learn what HTML is all about, and why you too should learn it.</p>

  <h3>What is HTML?</h3>
  <p>HTML stands for <strong>H</strong>yper<strong>T</strong>ext <strong>M</strong>arkup <strong>L</strong>anguage. It's
    a markup
    language that web
    developers use to structure and describe
    the content of a webpage (not a programming language).</p>
  <p>HTML consists of elements that describe different types of content: paragraphs, links, headings, images, video,
    etc. Web browsers understand HTML and render HTML code as websites.</p>
  <p>In HTML, each element is made up of 3 parts:</p>
  <ol>
    <li>The opening tag</li>
    <li>The closing tag</li>
    <li>The actual element</li>
  </ol>

  <p>You can learn more at <a href="https://developer.mozilla.org/en-US/docs/Web/HTML" target="_blank">MDN Web Docs</a>
  </p>

  <h3>Why should you learn HTML?</h3>
  <p>There are countless reasons for learning the fundamental language of the web. Here are 5 of them:</p>

  <ul>
    <li>To be able to use the fundamental web dev language</li>
    <li>To hand-craft beautiful websites instead of relying on tools like Worpress or Wix</li>
    <li>To build web applications</li>
    <li>To impress friends</li>
    <li>To have fun 😀</li>
  </ul>
```

```
  <p>Hopefully you learned something new here. See you next time!</p>
</body>

</html>
```

Structuring our Page

1. The elements just appear one after another but not grouped together into logical groups
    1. Container elements group elements together
2. Page navigation - groups links at the top of a page
    1. `<nav>` - navigation
        1. An invisible box which contains the elements
3. `<header>` - top part of a web-document
    1. Or top part of some smaller unit
4. `<article>` - can contain blog-post
    1. Could be things from the real-world
        1. Computer
        2. Pen
        3. ...
5. Why not call all of them a `<nav>`, say?
    1. For semantic HTML (?)
6. `<footer>` - comes at the end of the page
    1. `©` - HTML entity
        1. HTML Entities: In resources page

A Note on Semantic HTML

1. Sematic HTML:
    1. Certain elements have a meaning or purpose attached to them
        1. We need to think about what the element actually means and what it stands for
            1. Not just how it looks on the page
    2. Not all elements are semantic
    3. Example: `<div>`
        1. Box without any meaning
            1. Used before HTML5
            2. **Use it only when we don't want to attach any meaning to certain box element**
    4. `<p>` - has semantic meaning
        1. Browser gives spaces around `<p>` element

Installing Additional VS Code Extensions

1. Extensions:
    1. image preview
        1. Displays small image preview in the gutter
    2. color highlight
        1. CSS - for visualization
    3. Auto Rename Tag
        1. If change the name of an element, it automatically changes closing tag
    4. Settings:
        1. auto close tag
            1. Enable
    5. Live Server
        1. Automatically reloads the page

CHALLENGE #1

1. `<aside>` - secondary info
    1. Some related posts related to the `<article>` (the main part)
    2. Usually a side bar
    3. Why list?
        1. It is a list of related post - semantically

CHALLENGE #2

1. Codepen - online platform
    1. codepen.io/pen
    2. Register:
        1. We can get url that can be shared with others
2. https://codingheroes.io/resources
3. Image can come from a URL
4. `<button>`
5. Example:

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>E-Commerce</title>
</head>

<body>
```

```
    <article>
      <h2>Converse Chuck Taylor All Star Low Top</h2>
      <img src="images/challenges.jpg" alt="Trouser" width="250px" />
      <p><strong>$65.00</strong></p>
      <p>Free shipping</p>
      <p>Ready to dress up or down, these classic canvas Chucks are an everyday wardrobe staple.</p>
      <p><a href="https://www.converse.com" target="_blank">More information &rarr;</a></p>
      <h3>Product details</h3>
      <ul>
        <li>Lightweight, durable canvas sneaker</li>
        <li>Lightly padded footbed for added comfort</li>
        <li>Iconic Chuck Taylor ankle patch.</li>
      </ul>
      <button>Add to cart</button>
    </article>
  </body>

  </html>
```

Section 3: CSS Fundaments

Section Intro

1. CSS - for styling content we write in HTML
2. Basics
    1. Styles
    2. Theoretical concepts
        1. How CSS works

Introduction to CSS

1. What is CSS?

    1. Cascading Style Sheets
    2. It is one of the core technologies of a web-page
    3. CSS describes the **visual style and presentation** of the **content witten in HTML**
    4. CSS consists of nearly countless **properties** that developers use to format the content: properties about font, text, spacing, layout, etc.
        1. We use most important ones many times to make it intuitive

2. Example:

```
h1 {
    color: blue;
    text-align: center;
    font-size: 20px;
}
```

    1. `h1` - selector
        1. All `h1` elements
    2. `color: blue;` - **declaration (M)** (property: value)
        1. aka - style
    3. `color` - property
    4. `blue` - value
    5. `{...}` - **declaration block (M)**
    6. **CSS rule**: **selector + declaration block (M)**

Inline, Internal and External CSS

1. Three places to write CSS
    1. Inline
    2. Internal
    3. External
2. New project:
    1. 03-CSS-Fundamentals
3. Inline CSS:

    1. Writing CSS inside of the element

```
<h1 style="color: blue">...</h1>
```

        1. It should almost never be used
4. Internal CSS: (for something small and we want to maintain all the code in a single file)
    1. In `<head>`, we declare `<style>` element
        1. It decouples the style from HTML element
            1. Coupling is not good in programming
        2. Separation of concerns
    2. It bloats HTML file if we have large CSS code
        1. Solution: We put it in a special css file
5. External CSS:

    1. style.css

```
h1 {
    color: blue;
}
```

    2. index.html

```
<link href="style.css" rel="stylesheet" /> <!-- **(M)** -->
```

Stying Text

1. Properties to style text:

```
h1 {
  /* color: blue; */
  font-size: 26px;
  font-family: sans-serif;
  text-transform: uppercase;
  font-style: italic;
}

h2 {
  font-size: 40px;
  font-family: sans-serif;
}

h3 {
  font-size: 30px;
  font-family: sans-serif;
}

h4 {
  font-size: 20px;
  font-family: sans-serif;
  text-transform: uppercase;
  text-align: center; /* It is in the center of its parent element */
}

p {
  font-size: 22px;
  font-family: sans-serif;
  line-height: 1.5; /* 1.5 times the font-size */
}

li {
  font-size: 20px;
  font-family: sans-serif;
}
```

2. Inheritance
    1. Child elements get the style of parent elements if no style is specified for the child elements

Combining Selectors

1. Setting `font-family` is applied to each and every element. It is better to select all the elements and apply the property.

Class and ID Selectors

1. With ID:

```
#author {
  font-style: italic;
  font-size: 18px;
}

#copyright {
  font-size: 16px;
}
```

    1. We are not allowed to repeat id names
        1. We can use an id only once

2. Using Classes:

```
.related-author {
  font-size: 18px;
  font-weight: bold;
}
```

    1. If we apply multiple rules, all rules apply (but need more details on how they are applied)
    2. In the real-world we don't use ids, we use classes
        1. We are prepared for the future
            1. If we want to re-use, it is not be possible with id
                1. Class can be re-used
            2. Changing id to class, it can cause bugs with big projects

Working with Colors

1. Theory of representing colors using codes
    1. RGB model:
        1. Every single color can be represented by a combination of RED, GREEN, BLUE
        2. Each of the 3 base colors can take a value between 0 and 255
            1. Which gives 16.8 million different colors
        3. White:
            1. R: 255, G: 255, B: 255
        4. Black:
            1. R: 0, G: 0, B: 0
        5. Cyan:
            1. R: 0, G: 255, B: 255
        6. Yellow:
            1. R: 255, G: 255, B: 0
    2. Defining colors in CSS:
        1. RGB/RGBA notation:
            1. `rgb(0, 255, 255)`

2. `rgb(244, 179, 63)` - real looking color
3. `rgb(0, 255, 255, 0.3)` - contains alpha (transparency)
4. `rgb(244, 179, 63, 0.7)`
    1. We can generate using color picker
    2. We can copy the colors
2. Hexadecimal notation:
    1. Instead of using a scale 0 to 255, we go from 0 to ff (255 in hexadecimal numbers)
        1. `#00ffff`
        2. `#f4b33f`
    2. Shorthand
        1. `#00ffff` - `#0ff`
3. **In practice, we mostly use hex numbers but use `rgba` function if we need transparency**
4. Shades of Grey:
    1. When colros in all 3 channels are the same, we get a grey color
    2. There are 256 pure grays to choose from
        1. `rgb(0, 0, 0)` to `rgb(255, 255, 255)`
        2. `rgb(69, 69, 69)` - `#444444` - `#444`
        3. `rgb(183, 183, 183)` - `#b7b7b7`

2. Properties:

    1. If we use two definitions in a declaration block, the latest definition is used

3. Borders:

```
aside {
    /* ... */
    border: 5px solid #1098ad;
    /*
    border: 5px dotted #1098ad;
    border: 5px dashed #1098ad;
    border-top: 5px solid #1098ad;
    border-bottom: 5px solid #1098ad;
    */
}
```

    1. It is a **short-hand property**
        1. One property can be used to define different properties

Pseudo-classes

1. Styling first item in a list

```
li:first-child { /* li which is the first-child */
    font-weight: bold;
}

li:last-child {
    font-style: italic;
}

li:nth-child(2) {
    color: red;
}

li:nth-child(odd) {
    color: red;
}
```

    1. If we mix elements, the pseudo-classes do not work very well
    2. **They are perfect for child elements that are all the same**

Styling Hyperlinks

1. Targetting elements with `href` attribute only

```
a:link {
    color: orangered;
    font-weight: bold;
    text-decoration: underline wavy orangered;
}

a:active { /* When we click a link, the active pseudo class is added to the element and we can select it */
    background-color: black;
    font-style: italic;
}
```

Using Chrome DevTools

1. Google chrome dev-tools
    1. Opening Dev-Tools
        1. Right-click > Inspect
        2. View > Developer > Developer Tools
        3. Ctrl
    2. Elements
        1. If we hover, the elements on the page will get highlighted
        2. Styles - tab
            1. Styles are displayed
            2. We can hover to highlight
    3. We can fake a state by clicking:
        1. :hov
            1. Select :hover
                1. Shows rule that applies

4. Right click on element and inspect

CSS Theory #1: Conflicts Between Selectors

1. What if there are multiple css rules?
    1. Example:

```
<p id="author-text" class="author">
    ...
</p>

.author {
    font-style: italic;
    font-size: 18px;
}

#author-text {
    font-size: 20px;
}

p,
li {
    font-family: sans-serif;
    color: #444444;
    font-size: 22px;
}
```

        1. All of them apply
            1. If there are conflicting declarations:
                1. `font-size`
                2. Rules applied to resolve conflicts:

```
Highest Priority: Declarations mared ! important (as a last resort to resolve conflict)
    |
    If No important!
    |
    v
    Inline style (`style` attribute in HTML)
    |
    If no inline styles
    |
    v
    ID (#) selector (if multiple, the last rule is applied, advanced concepts - cascades, selector-specificity)
    |
    If No # selector
    |
    v
Class (.) or pseudo-class (:) selector
    |
    If No . or : selector
    |
    v
Element selector (p, div, li, etc)
    |
    If No element selector
    |
    v
Lowest Priority: Universal selector (*)
```

                    1. Inspect: Shows strike through on rules that didn't apply
                    2. Pseudo classes also have priority over elements
                    3. Example:

```
a:link {
    /* higher priority */
}

a {
    /* lower priority */
}

footer p {
    color: green !important; /* highest priority */
}
```

                        1. Solution: Write simple selectors
                            1. Not too much nesting
                            2. Not too many ids and classes in the same selector

CSS Theory #2: Inheritance and the Universal Selector

1. An element inside another element gets the style of the parent element.
    1. Example:

```
body {
    color: red;
}
```

        1. color in child element overrides the property inherited from the parent
        2. **Inherited properties have the lowest priority**
            1. body does not select all elements but elements inside body will inherit some of the properties

1. Not all properties get inherited. It's mostly the ones **related to text**:
    1. `font-family`
    2. `font-size`
    3. `font-weight`
    4. `font-style`
    5. `color`
    6. `line-height`
    7. `letter-spacing`
    8. `text-align`
    9. `text-transform`
    10. `text-shadow`
    11. `list-style`
    12. etc.
2. Example:

```
<body>
    <nav>
        This is the navigation
    </nav>

    <h1>My website</h1>

    <p>
        The text is the paragraph
        is completely irrelevant
    </p>
</body>
```

```
body {
    color: #444444;
    font-size: 16px;
    font-family: sans-serif;

    border-top: 10px solid #1098ad;
}

h1 {
    color: #1098ad; /* Overrides */
    font-size: 32px; /* Overrides */
    text-transform: uppercase;
}
```

    1. Universal selector applies to all elements but inheritance is not applied
        1. Useful if inheritance does not work for certain properties

CHALLENGE #1

```
body {
  font-family: sans-serif;
  line-height: 1.4;
}

.article {
  border: 4px solid black;
}

h1,
h2,
h3 {
  text-transform: uppercase;
}

h2 {
  background-color: #f7f7f7;
  text-align: center;
  font-size: 22px;
}

.details-list {
  list-style: square;
}

.money {
  font-size: x-large;
}

.btn {
  background-color: black;
  color: white;
  text-transform: uppercase;
  border: none;
  font-size: 20px;
}

.btn:hover {
  background-color: white;
  color: black;
  cursor: pointer;
}

.imp-note {
  text-transform: uppercase;
  font-size: x-small;
  font-weight: bold;
  color: #777;
}

.more-info {
```

```
  color: black;
}

.more-info:hover,
.more-info:active {
  text-decoration: none;
}
```

CSS Theory #3: The CSS Box Model

1. CSS Box Model:
    1. Defines how elements are displayed on a web-page and how they are sized
        1. Each element on a web-page can be seen as a rectangular box
            1. Each of the boxes can have:
                1. Content
                2. Border
                3. Space inside
                4. Space outside

Content

1. Actual content
    1. Text, images, video, table, etc.
2. We can specify both **height** and **width** of the content area

Border

1. A line around the element
2. It is still inside of the element

Padding

1. Invisible whitespace we can define around the content, inside the element
    1. It is between border and content

Marging

1. It defines space outside the element (between elements)

Notes

1. All the elements are optional
    1. We don't have to specify padding, margins, or borders
2. Fill Area:
    1. Text content or images are inside content box
    2. Background images or background color are applied to the entire fill area:
        1. Content + Padding + Border
3. Example:
    1. Drawing - content area
    2. White paper around - padding
    3. Frame - border
    4. Space outside of the frame (between frames) - margin

Element Height and Width Calculation

1. We can specify the height and width of the content
    1. If we don't specify, the box model will imply them based on the content
        1. They are not the final sizes
2. **Final element width = left border + left padding + width + right padding + right border**
3. **Final element height = top border = top padding + height + bottom padding + bottom border**
4. Marging is not part of the width and height calculations of the element
5. We can specify the above values using CSS properties
6. This is the default behavior which can be changed

Using Margins and Paddings

1. Padding:
    1. Space inside of the element

```
padding: 20px 10px
```

        1. first value: top & bottom
        2. secodn value: left & right

2. Margin:
    1. Space between elements

```
li {
  font-size: 20px;
  margin-bottom: 10px;
}

li:last-child {
  margin-bottom: 0; /* No unit */
}
```

3. Global reset:
    1. Set all margins and paddings to zero because it is hard to work with existing margins and padding of all the elements on a page.
    2. How to do it?

```
* {
    margin: 0;
    padding: 0;
}
```

        1. Very common

    3. Vertical spacing:
        1. It is common to define space at the bottom
            1. Stick to `margin-bottom` or `margin-top` and don't mix them
    4. Consider the following scenario:

```
p {
    margin-bottom: 15px;
}

h3 {
    margin-top: 40px;
}
```

        1. `p` is above an `h3` element
        2. The space between `p` and `h3` is not 15px + 40px = 55px. It is only 40px (the space between the elements)
            1. This is called **collapsing margins**
                1. Only the larger of the two is visible

Adding Dimensions

1. It is not straight forward to center vertically
2. We specify dimentions of images almost always in css and not in HTML

Centering our Page

1. How to center a page inside a browser?
    1. Put all the content in a container

CHALLENGE #2

```
* {
    margin: 0;
    padding: 0;
}

.product {

}
```

```
1. `margin-bottom` doesn't work for links
    1. We cannot use margins on
```

CSS Theory #4: Types of Boxes

1. Many elements occupy as much space as they can
2. Some elements occupy the space that its content takes (inline boxes)
    1. `img`
    2. `strong`
    3. `a`
3. Block level elements / boxes:
    1. They occupy all the space they can and they add a line break at the end
        1. They cannot be side-by-side of one-another
    2. They are formatted visually as Blocks
    3. **The elements occupy 100% of parent element's width no matter the content**
    4. The elements are stacked vertically by default
        1. One after another
    5. The box-model applies as showed earlier
        1. It is not true for inline elements
    6. By default most elements are block level elements
        1. `body`
        2. `main`
        3. `header`
        4. `footer`
        5. `section`
        6. `nav`
        7. `aside`
        8. `div`

9. `h1-h6`
10. `p`
11. `ul`
12. `ol`
13. `li`
14. etc.

7. **We can change inline boxes to block level boxes**

```
display: block; /* **(M)** */
```

8. Inline elements:
    1. They occupy just the size of the content
    2. They don't append a line-break after the element
    3. It doesn't make sense if they occupy the entire line
    4. **Height and width do not have any effect**
    5. **Padding and margin are only applicable horizontally**
        1. left and right only
    6. Default inline elements:
        1. `a`
        2. `img`
        3. `strong`
        4. `em`
        5. `button`
        6. etc.
    7. Changing an element from block level to inline level:

```
display: inline; /* **(M)** */
```

9. Example:

```
nav a:link {
  background-color: orangered;
  margin: 20px; /* top and bottom margins are unchanged - box model shows but did not actualize any space */
  padding: 20px; /* Space is created only inside an element but `a` is the same */
  display: block; /* All elements stack one above another and occupy full width */
}
```

10. We don't set fixed height on elements
    1. Because the element will not expand to fit the content

4. **Inline Block Elements**
    1. It is a mix of inline and block level
        1. Block-Level Boxes
        2. Inline Boxes
            1. Looks like inline from the outside, behaves like block-level on the inside
                1. **Box model applies as it does with block-level boxes**
        3. Inline-Block Boxes
        4. Occupies only content's space
        5. Causes no line-breaks
        6. Box-model applies as shown
        7. How to set:

```
display: inline-block; /* **(M)** */
```

            1. Occupies only content size but behaves like block level element
        8. **Images behave like inline block elements**

CSS Theory #5: Absolute Positioning

1. Complex part
2. Normal flow vs Absolute Positioning
    1. Normal Flow:
        1. Default positioning of elements on the page
            1. How to set?

```
position: relative
```

        2. Elements are laid out according to their order in the HTML code
    2. Absolute Positioning

1. Element is removed form the normal flow: "out of flow"
    1. Such element loses impact on surrounding elements
        1. Might overlap
2. **To position an element with absolute positioning, we can use the following from its relatively positioned container**
    1. `left`
    2. `right`
    3. `top`
    4. `bottom`
3. How to position:

```
position: absolute; /* **(M)** */
```

    1. It is out of flow
4. Example:
    1. **Emojis**:

```
ctrl + space, windows + .
```

5. To position an element absolutely in relation to another element, it must be inside that element.

```
+------
|
|              +--
|              | .el {
            position: absolute;
            top: 100px;
            left: 200px;
            background-color: #f4b33f;
            }

| .container {
.   position: relative;
.   background-color: #f7e6c1;
. }
+---
```

    1. **It is the first parent element that is set to relative from which the absolutely positioned element will be set**

```
body {
    /* ... */
    position: relative;
}

.post-header {
    position: relative;
}

button {
    position: absolute; /* will be positioned relative to the first parent above this element, which is .post-header */
}
```

    2. This positioning is not used to build complex layouts
        1. It is used for small elements like buttons etc.

Pseudo-elements

1. They are elements that do not exist in the HTML but that we can still select and style in CSS
    1. Example:
        1. First letter in a paragraph
        2. First line of a paragraph
        3. etc.
2. First letter:

```
h1::first-letter {
    font-style: normal;
    margin-right: 5px;
}
```

3. First line:

```
p::first-line {
    color: red;
}
```

4. Siblings:
    1. Elements that have the same parent element
    2. Adjacent sibling: The next sibling
    3. How to select?

```
h3 + p::first-line { /* Only the paragraphs that come after h3 are selected */
    color: red;
}
```

5. Most used pseudo elements:

1. `after` **(M)**
    1. First child of the selected element (this will become the last child of the element)
    2. Useful for small cosmetic style (without adding a new element)
    3. **By default, any pseudo element is an inline element**
    4. Example:

```
<h2>
    The Basic Language of the Web: HTML
    ::after
</h2>
```

2. `before` **(M)**
    1. This will become the first child of the element

```
<h2>
    ::before
    The Basic Language of the Web: HTML
</h2>
```

Developer Skill #1: Googling and Reading Documentation

1. Techniques to Google stuff and how to read documentation about properties we don't know or how we don't know to use:
    1. Google: css property to add mouse cursor to button
        1. stackoverflow.com - good
        2. css-tricks.com - good
        3. developer.mozilla.org - good
    2. Google: css how to center anchor elements
        1. stackoverflow.com

```
<div class="my-class">
    <a ...>
</div>

.my-class {
    text-align: center;
}
```

        2. Google: mdn css text-align
            1. Description and demos
                1. text-align CSS property sets the horizontal alignment of the **content inside** a block element or table-cell
                2. Browser compatibility for the CSS property
        3. Google: mdn css border
            1. Examples

Developer Skill #2: Debugging and Asking Questions

1. Debugging and asking questions if we cannot solve certain problems
    1. Bug - an error or mistake
        1. Debugging: Finding and fixing errors in code
2. Most common bug in HTML:
    1. Some elements remain unclosed
        1. copy-paste
        2. Deletion
        3. If all text is bold
            1. Look where the code is not bold
                1. Find the location and look there
    2. Close a tag and have content outside
        1. Go to the place where this problem exists

```
<li></li>gsfsdfsd</li>
```

    3. If error is not apparent
        1. We figure out the bug only later on when we write css
            1. Solution: Use HTML validator tool
                1. Copy the code
                2. Google: HTML validator
                    1. Paste
    4. Diff-Checker
        1. Compare two codes
3. Common Bugs in CSS:
    1. Conflicts between selectors
        1. Example:

```
.main-header nav a:link {
    margin-top: 100px;
```

```
}

nav a:link {
    margin-top: 10px;
}
```

1. **More complex rule applies many of the times**
    1. Avoid writing complex rules
        1. Check specificity: Higher the numbers, the more the sectors will apply
2. Misspelled property name
    1. Solution: Inspect and check out
        1. Select pseudo-element if that has problem
            1. Unknown property name
        2. We can experiment with code in dev-tools
            1. Tweek values
                1. Up and down keys
                2. Shift + Up or Down (increments in steps of 10)
3. Path to CSS folder is not correct
    1. If CSS is not applied at all
4. If bugs cannot be fixed
    1. Search in stackoverflow
    2. Post question in udemy
        1. Use codepen
            1. Copy the url and send it

CHALLENGE #3

1. Solution:

```
.product {
    ...
    position: relative;
}

.sale {
    background-color: #ec2f2f;
    color: #fff;
    font-size: 12px;
    text-transform: uppercase;
    font-weight: bold;
    padding: 7px 15px;
    display: inline-block;
    letter-spacing: 2px;
    /* width: 40px;
    text-align: center; */
    position: absolute;
    top: -17px;
    left: -38px;
}
```

Section 4: Layouts: Floats, Flexbox, and CSS Grid Fundaments

Section Intro

1. Layouts
    1. How to build layouts using
        1. Floats
        2. Flexbox
        3. CSS Grid
    2. One of the most important sections

The 3 Ways of Building Layouts

1. What does layout mean?
    1. Layout is the way text, images and other content is placed and arranged on a webpage
        1. Elements are arranged in a certain way
    2. Layout gives the page a visual structure, into which we place our content
    3. Building layout
        1. Arranging elements into a visual structure instead of placing them just one after another
    4. Purpose:
        1. Makes sites easier to understand
        2. Makes sites better looking

Page Layout vs Component Layout

1. Page Layout
    1. Laying out elements that are the big pieces of content inside a web page or website
    2. The bigger page layouts are made up of components
    3. The components themselves need to have some layout
        1. The components themselves are made up of smaller pieces of content which need to be arranged in a particular way
2. Component Layout
    1. Components within a page layout also have a layout themselves
3. How to build layouts using CSS?
    1. There are 3 ways of building layouts with CSS:

Float Layouts

1. The old way of building layouts of all sizes, using `float` CSS property. Still used (especially in older websites), but getting outdated fast
    1. Newer technologies:

        1. Flex-box
        2. CSS Grid
    2. This is important to learn especially if we need to work on an older website

Flexbox

1. It is one of the modern ways of laying out elements in a **1-dimensional row** without using floats. Perfect for **component layouts**
    1. They might be perfect for simpler component layouts

CSS Grid

1. For laying out elements in a fully-fledged **2-dimensional grid**. Perfect for **page layouts and complex components**.
    1. Perfect for big page layouts and complex components we saw

Using Floats

1. How does it work?
2. Example: New starter files
    1. What if we want two elements to appear side by side
        1. One way: inline or inline-block
        2. Another way: `float`
    2. The floated elements are removed from the page and the parent element will lose its children.
        1. If all children are floated, the parent has no more content
            1. Height = 0
        2. This is called **collapsing elements**
3. Comparison:
    1. Element is removed from the normal flow: "out of flow"
        1. However, they still have impact on the surrounding elements
            1. Text and inline elements will wrap around the floated element (unline absolute positioning)
    2. The container will not adjust its height to the element (collapsing)

Clearing Floats

1. One way:

```
</nav>
<div class="clear"></div> <!-- clutter -->

.clear {
  clear: both;
}
```

2. Second way: Clear-fix hack

```
<header class="main-header clearfix">

.clearfix::after {
  clear: both;
  content: "";
  display: block;
}
```

Building a Simple Float Layout

1. Main parts of the layout:
    1. Header
    2. Article
    3. Aside
    4. Footer

box-sizing: border-box

1. Another version of the box-model
    1. Can we define total width instead of just content width?

```
box-sizing: border-box; /* **(M)** */
```

        1. width: border to border (content width + padding)
        2. height: border to border (content height + padding)
        3. Default: `content-box`

CHALLENGE #1

Introduction to Flexbox

1. Start with starter files
    1. 04-CSS-Layouts
        1. flexbox.html
        2. css-grid.html

A Flexbox Overview

1. `.container`
    1. To use flexbox, we just have to set the `display` property to `flex` on some container element

1. container element: any element that has child elements

2. By default the elements are side-by-side

3. **Items of flexbox**: child elements of a flex container

    1. Horizontally: **Each item occupies exactly the size it needs for its content**

    2. Vertically: **Each item is as tall as the tallest item**

```
.el--3 {
  background-color: green;
  height: 150px;
}
```

        1. **Otherwise they occupy the space required for the content**

4. Example:

```
.container {
  /* STARTER */
  font-family: sans-serif;
  background-color: #ddd;
  font-size: 34px;
  margin: 40px;

  /* FLEXBOX */
  display: flex;
  align-items: center;
  /* Aligns items vertically */
  /* align-items: flex-start; */
  /* align-items: flex-end; */
  /* align-items: stretch; */
  /* default */
  justify-content: center;
  justify-content: space-between;
  /* the remaining space is distributed among all the elements  */
}
```

Spacing and Aligning Flex Items

1. Topics:
    1. What is flexbox?
    2. When and why do we need it?
    3. Important terminology around flexbox
    4. Overview of all properties that are part of the flexbox spec

What is Flexbox?

1. Flexbox is a set of related **CSS properties** for **building 1-dimensional layouts**
    1. The main idea behind flexbox is that empty space inside a container element can be **automatically divided** by its child elements
        1. Browser automates the work that web-developers were previously doing
2. Flexbox makes it easy to automatically **align items to one another** inside a parent container, both horizontally and vertically
3. Flexbox solves common problems such as (hard problems before)
    1. Vertical centering
    2. equal-height columns
4. Flexbox is perfect for **replacing floats**, allowing us to write fewer lines and cleaner HTML and CSS code
    1. Easy to maintain websites over a long run
    2. To write less bugs

Flexbox Terminology

1. **Flex container**: **(M)**

    1. The element on which we want to use flexbox is called a flex container

    2. How to make an element a flex container?

```
display: flex;
```

        1. After setting this property, all the direct children inside the flex container become **flex items (M)**

    3. We can use different properties on flex container and flex items

    4. **Main axis**: **(M)** The direction in which the flex items are laid out

    5. **Cross axis**: **(M)** The perpendicular direction to the main axis

    6. We can align elements along the main axis and along the cross axis

Flex Container Properties

1. `gap: 0 | <length>` **(M)**
    1. To define **space between items**, without using margin
2. `justify-content: flex-start | flex-end | center | space-between | space-around | space-evenly` **(M)**
    1. To align items along main axis (**horizontally**, by default)
3. `align-items: stretch | flex-start | flex-end | center | baseline` **(M)**
    1. To align items along cross axis (**vertically**, by default)
        1. Print out a slide and keep it as a summary sheet
4. `flex-direction: row | row-reverse | column | column-reverse` **(M)**
    1. To define which is the **main axis**

5. `flex-wrap: nowrap | wrap | wrap-reverse` **(M)**
    1. To allow items to wrap into a new line if they are too large
6. `align-content: stretch | flex-start | flex-end | center | space-between | space-around` **(M)**
    1. Only applies when there are **multiple lines** (`flex-wrap: wrap`)

Flex Items Properties

1. `align-self: auto | stretch | flex-start | flex-end | center | baseline` **(M)**
    1. To override `align-items` for individual flex items
2. `flex-grow: 0 | <integer>` **(M)**
    1. To allow an element **to grow** (0 means no, 1+ means yes)
3. `flex-shrink: 1 | <integer>` **(M)**
    1. To allow an element **to shrink** (0 means no, 1+ means yes)
4. `flex-basis: auto | <length>` **(M)**
    1. To define an item's width, **instead of the width** property
5. `flex: 0 1 auto | <int> <int> <int>` **(M)**
    1. Recommended shorthand for flex-grow, -shrink, -basis
6. `order: 0 | <integer>`
    1. Controls order of items. -1 makes item first, 1 makes it last

Implementation

1. `align-self` - for individual items

```
.el--1 {
    align-self: flex-start;
}

.el--5 {
    align-self: stretch;
}
```

2. `order` - adapting a bigger layout to smaller layout say
    1. Default value for all elements: 0
    2. To move an element to first position, give a number lower than 0

```
.el--1 {
  align-self: flex-start;
  order: 2;
}

.el--5 {
  align-self: stretch;
  order: 1;
}

.el--6 {
  order: -1;
}
```

3. Manually defining space betwee flex items:
    1. Two ways:
        1. Adding margin:

```
.el {
    margin-right: 40px;
}
```

            1. It becomes a pain to remove the margin from the last element

        2. Using `gap`

```
.container {
    // ...
    gap: 30px;
}
```

The Flex Property

1. `flex` is shorthand for `flex-grow`, `flex-shrink` and `flex-basis`

```
.el {
    /* defaults */
    /*
    flex-grow: 0;
    flex-shrink: 1;
    flex-basis: auto;
    */

    flex-basis: 100px; /* width, we don't use width property */
}
```

    1. But if content is > 100px, the element extends to fit the element content
        1. It is only a recommendation made to browser
            1. Browser makes decision to
                1. Shrink the elements to fit the container
                    1. If there is not enough space in the container to fit the items, browser will shrink to fit the items (because `flex-shrink: 1` by default

1. `flex-shrink` determines whether the flexbox is allowed to shrink elements if necessary or not.
2. `flex-grow` determines whether the flexbox is allowed to grow elements as large as they can grow if necessary or not

```
.el {
  /* flex-basis: 200px; */
  flex-shrink: 0;
  flex-grow: 1; /* Remaining space is divided among elements */
}
```

2. Expand the elements to fit the content
3. What if only one of them is set to 1?

```
.el--1 {
    // ...
    flex-grow: 1;
}
```

    1. Only the current element is allowed to fill out the remaining space

4. If all elements have the same number, they would have the same size

    1. If one of them was a higher number, it would get proportional size
    2. `flex` is recommended to the three properties

Adding Flexbox to Our Project

1. Doing using flexbox:
2. Flex style is applied to the parent of the containing elements
3. Example:

```
/* FLEXBOX */
.main-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
}

.author-box {
  display: flex;
  align-items: center;
  margin-bottom: 15px;
}

.author {
  margin-bottom: 0;
  margin-left: 15px;
}

.related-post {
  display: flex;
  align-items: center;
  gap: 20px;
  margin-bottom: 30px;
}

.related-posts:last-child {
  margin-bottom: 0px;
}

.related-link:link {
  font-size: 17px;
  font-weight: bold;
  font-style: normal;
  margin-bottom: 5px; /* doesn't work */
  display: block;
}

.related-author {
  margin-bottom: 0;
  font-size: 14px;
  font-weight: normal;
  font-style: italic;
}
```

Building a Simple Flexbox Layout

1. With flexbox, it is necessary to define a new flex container for layouts

CHALLENGE #2

1. My Solution:

```
/* FLEXBOX */
.product-container {
  display: flex;
  gap: 40px;
  align-items: center;
  justify-content: space-between;
}

.price-container {
  display: flex;
  align-items: center;
  justify-content: space-between;
```

```
  flex: 0 0 250px;
}

.price-details-container {
  flex: 0 0 250px;
}

.product-details-container {
  flex: 0 0 250px;
}

.color-container {
  display: flex;
  gap: 20px;
}
```

2. Instructor's solution:

Introduction to CSS Grid

1. New project files
    1. Click **CSS Grid**
2. CSS Grid:
    1. The most modern way of building layouts and the most complete one
    2. It is the easiest way as well
3. Parts:
    1. Grid containers
        1. How to conver an element to a grid contianer:

           ```
           display: grid; /* **(M)** */
           ```

        2. To define our two-d layout
            1. Columns

               ```
               grid-template-columns: 250px 150px; /* We can define as many width values as we want. For each of the values, one column will b
               ```

                1. All the elements will get displayed in columns and rows (left to right and top to bottom)
                    1. Rows will get created as necessary

            2. Rows

               ```
               grid-template-rows: 300px 200px;
               ```

    2. Grid items
        1. Elements stretch the entire cell (just like in flex-box)
            1. Across height and width we defined for each cell
                1. Areas between columns and rows
            2. If height is defined for any eliment, the entire row will have that height
                1. The other elements stretch (by default) to fill the row
        2. `gap` - the only way to give gap
            1. Do not use margins

               ```
               gap: 30px; /* Both between columns and rows */
               column-gap: 30px; /* Only for columns */
               row-gap: 60px; /* Only for rows */
               ```

                1. Previously `grid-gap`

A CSS Grid Overview

1. What is CSS Grid?
    1. CSS Grid is a set of **CSS properties** for **building 2-dimensional layouts**
    2. The main idea behind CSS Grid is that we **divide a container element into rows and columns** that can be filled with its child elements
        1. Spanning elements over multiple rows and columns
        2. Overlap different elements
        3. etc.
    3. Advanages:
        1. It two-dimensional contexts, CSS Grid allows us to write **less nested HTML** and **easier-to-read CSS**
    4. CSS Grid is **not meant to replace flexbox!**. Instead, they word perfectly together. Need a **1D** layout? Use flexbox. Need a **2D** layout? Use CSS Grid.

Basic CSS Grid Terminology

1. Grid container
    1. Convert an element into a grid container as follows:

       ```
       display: grid
       ```

2. Grid items
    1. Child elements of a grid container
3. Row axis
    1. We cannot change the directions of row and column axes
4. Column axis

    1. We cannot change the directions of row and column axes

5. Grid lines

    1. Lines that divide the grid and separate the columns and the rows
    2. Numbered:
        1. 1 - (number of rows + 1)
        2. 1 - (number of columns + 1)
    3. The numbers can be used to place a grid item exactly in a particular place in the grid

6. Grid cell (might be filled by a grid item or not)

    1. The spaces created by intersecition of the grid lines
    2. Each grid cell doesn't have to be

7. Gutters (gaps)

    1. Spaces between grid items

```
column-gap
row-gap
```

8. Grid track (row/column)

    1. Space occupied by Grid items taken together in a row (excluding the gutters)
        1. Similar for columns

Grid Container Properties

1. `grid-template-rows: <track-size>*`
2. `grid-template-columns: <track-size>*`
    1. Used to establish the grid **row and column tracks**. One length unit for each track
    2. Any unit can be used
    3. New `fr` fills unused space
3. `row-gap: 0 | <length>`
4. `column-gap: 0 | <length>`
5. `gap: 0 | <length>`
    1. To construct empty space between tracks
6. `justify-items: stretch | start | center | end`
7. `align-items: stretch | start | center | end`
    1. To align items inside rows / columns (horizontally / vertically)
8. `justify-content: start | center | end | ...`
9. `align-content: start | center | end | ...`
    1. To align entire **grid inside grid container**.
    2. Only applies if container is larger than the grid

Grid Items Properties

1. `grid-column: <start line> / <end line> | span <number>`
2. `grid-row: <start line> / <end line> | span <number>`
    1. To **place grid item** into a specific cell, based on line numbers. `span` keyword can be used an item across more cells
3. `justify-self: stretch | start | center | end`
4. `align-self: stretch | start | center | end`
    1. To **overwrite** `justify-items`/ `align-items` for single items

Sizing Grid Columns and Rows

1. `fr` unit

    1. Allows creation of flexible columns and flexible rows

```
grid-template-columns: 200px 200px 1fr 1fr;

grid-template-columns: 2fr 1fr 1fr 1fr; /* 2/5th, 1/5, 1/5, 1/5, 1/5

grid-template-columns: 3fr 1fr 1fr 1fr: /* 3/6th = 1/2 ... */

grid-template-columns: repeat(4, 1fr);

grid-template-columns: repeat(3, 1fr);
```

        1. Explicit rows: The three rows explicitly defined
            1. This is a good practice always and avoid implicit rows
        2. Implicit row: The row that is additional to the explicitly defined rows
            1. Row added automatically

    2. Height of items in a row is equal to the tallest item in the row by default

Placing and Spanning Grid Items

1. Sometimes it is important to place an element in a grid-cell other than the predefined one

2. Reset

    1. `height: 150px`
    2. `column-gap: 20px`

3. We can see grid details in dev-tools

4. Explicitly specifying position of a grid item:

```
.el--8 {
  grid-column: 2 / 3;
  grid-row: 1 / 2;
}

.el--2 {
  grid-column: 1 / 2; /* 1 / 3 - spans two columns */
  grid-column: 1 / span 3; /* spans 3 cells */
  grid-row: 2 / 3; /* We can ommit the second value is just one more than the first value */
}
```

1. The grid has negative numbers from right to left (-1 to -n)

Aligning Grid Items and Tracks

1. Move `display: none` to `container--1`
2. It is different:
    1. We can align tracks inside the container
    2. We can align grid items inside tracks
        1. If grid items are smaller than the cells they are in
3. Aligning grid tracks inside of the grid container (if grid is smaller than the grid container)
4. Example:

```
.container--2 {
  /* The element is completely removed from the page */

  /* STARTER */
  font-family: sans-serif;
  background-color: black;
  font-size: 40px;
  margin: 100px;

  width: 1000px;
  height: 600px;

  /* CSS GRID */
  display: grid;
  grid-template-columns: 125px 200px 125px;
  grid-template-rows: 250px 100px;
  gap: 50px;

  /* Aligning tracks inside container: distribute empty space */
  justify-content: center;
  /* exactly as in the flexbox */
  /* justify-content: space-between; */
  align-content: center;
  /* align-content: space-between; */

  /* Aligning items INSIDE cells: moving items around inside the cells */
  align-items: center;
  /* aligns items inside the cells - centers elements vertically: `stretch` is default */
  /* align-items: end; */
  justify-items: center;
  /* centers horizontally: `stretch` is default */
  /* justify-items: end; */
}

.el--3 {
  align-self: end;
  justify-self: end;
}
```

Building a Simple CSS Grid Layout

1. Blog-post layout
    1. We want to keep using flexbox in component layouts
        1. Especially if they are one-dimentional layouts
    2. We want to use css-grid for 2D page layout

CHALLENGE #3

Section 5: Web Design Rules and Framework

Section Intro

1. Web-design
    1. Special design framework
        1. A collection of design rules and guidelines that we can apply to different design ingredients
            1. Typography
            2. Color
            3. Spacing
            4. Images
            5. ...
        2. Practical guidelines
            1. To turn a developer into a professional who can design beautiful and professional looking UIs
                1. There are not many people who are good at both writing code and designing

Project Overview

1. Starter files: 05-Design (new files)
    1. All the HTML is available
    2. We will only be working on CSS and with some new properties
2. `<section>` - HTML element
3. `<span>` - generic inline element
4. `<blockquote>` - to quote someone

Overview of Web Design and Website Personalities

1. Web Design vs Web Development
    1. Web **designers** construct the overall **look and feel** of a website
    2. Web **developers** implement the design using **HTML, CSS, and JavaScript code**

Why Take Design Seriously

1. Good Design
    1. An immediate and lasting **good impression** of the brand or product.
    2. Makes the user **trust** the brand right away
    3. Increases the user's **perceived value** of the brand or product
    4. Gives users exactly **what they were looking for** when coming to the site, e.g. purchasing a product or finding information.
        1. Example: A website on TV must give as much revelvant information as required as fast as possible
2. Bad Design
    1. Makes users believe the brand doesn't really care about their product or service.
    2. Makes the user insecure about trusting the brand
    3. Makes the brand or product seem "cheap"
    4. Leaves users confused, and makes it hard for them to reach their goal

Anyone Can Learn Good Design

1. Good web design is **not subjective or creative**
2. Everyone can learn basics by following a **framework/system**
    1. Learn some rules and guidelines
3. Framework
    1. 100s of well-designed sites **deconstructed**
    2. Distilled into easy-to-learn and easy-to-apply **rules**
    3. Divided in 9 different areas of design: **ingredients**
    4. Rules will be applied based on **website personality**

Web Design Ingredients You Will Learn About

1. Ingredient: Typography
    1. About formatting and designing text
2. Ingredient: Colors (imp)
3. Ingredient: Images / Illustrations (imp)
    1. How to use images in an effective way
4. Ingredient: Icons
5. Ingredient: Shadows
6. Ingredient: Border-radius
    1. It is not insignificant
        1. Different border radius will give completely different look
7. Ingredient: Whitespace
8. Ingredient: Visual Hierarchy
9. Ingredient: User Experience
10. Ingredient: Components/ Layout
    1. How to use different components and types of layouts to represent different kinds of info

Website Personality

1. Design decisions for each of the ingredients will be taken based on the **website personality**
2. Overview of Website Personalities:
    1. **Serious/Elegant**: For luxury and elegance, based on think serif typefaces, golden or pastel colors, and big high-quality images
    2. **Minimalist/Simple**: Focusses on the essential text content, using small or medium-sized sans-serif black text, lines, and few images and icons
    3. **Plain/Neutral**: Design that gets out of the way by using neutral and small typefaces, and a very structured layout. Common in big corporations (Microsoft, Adobe, ...)
    4. **Bold/Confident**: Makes an impact, by featuring big and bold typography, paired with confident use of big and bright colored blocks
    5. **Calm/Peaceful**: For products and services that care (about users), transmitted by calming pastel colors, soft serif headings, and matching images/illustrations
    6. **Startup/Upbeat**: Widely used in startups, featuring medium-sized sans-serif typefaces, light-grey text and backgrounds, and rounded elements
    7. **Palyful/Fun**: Colorful and round designs, fueled by creative elements like hand-drawn icons or illustrations, animations, and fun language

Web Design Rules #1: Typography

Some Concepts First

1. Typography:
    1. Typography is the art and technique of arranging type to make written language **legible**, **readable**, and **appealing** when displayed. - Wikipedia
        1. It is making text beautiful and easy to read
2. **Serif vs Sans-Serif**
    1. Serif typeface:
        1. There are tails at the end of the lines (they are called serif)
        2. For tranditional/ classic looking feel
        3. Conveys trustworthiness (for user)
        4. Good for long text (article, online magazine)
    2. Sans-serif typeface: (easier to choose)

        1. They do not have tails at the end of the lines
        2. They have modern look and feel
        3. Clean and simple
        4. **Easier to choose for beginner designer!**
3. **Use Good Typefaces**
    1. Use only good and popular typefaces and play it safe (do not choose crazy or weird looking typefaces)
        1. Good examples of **sans-serif** typefaces (available on Google Fonts, Font Squirrel) - they change the look of websites (the following are for free)
            1. **Inter**
            2. **Open Sans**
            3. **Roboto**
            4. **Montserrat**
            5. **Work Sans**
            6. **Lato**
        2. Good examples of **Serif** typefaces
            1. **Merriweather**
            2. **Aleo**
            3. **Playfair Display**
            4. **Cormorant**
            5. **Cardo**
            6. **Lora**
4. **It's okay to use just one typeface per page! If you want more, limit to 2 typefaces** (otherwise the design might look ameteurish and strange)
5. **Choose the right typeface according to your website personality**
    1. Choose the right personality for your website (more on this later)
    2. Decide between a serif and sans-serif typeface
    3. Experiment with all the "good" typefaces (and other typefaces from Google Fonts!) to see which ones best fits your website's message (this will come with experience)
        1. If there is more content on the page, it becomes easier to see which one best fits the message
    4. You can keep trying different typefaces as you design and build the page
6. When choosing font-sizes, **limit choices**! Use a **type scale tool** or other **pre-defined range**
    1. Adapted from a Minor Third (1.2) type scale with base 10px
    2. Makes it easier to choose
7. Use a font size between **16px and 32px** for **normal** text
    1. Navigation: 16px
    2. Other text: 18px (print the slides)
8. For **long text** (like a blog post), try a size of 20px or even bigger
    1. If a user needs to read (helps if text is a little bigger)
        1. 20px
        2. 24px
        3. 32px
9. For **headlines**, you can go really big (50px+) and bold (600+), **depending on personality**
    1. 85px, 700 (boldness)
    2. 64px, 700 (boldness)
    3. 42px
10. For any text, don't use a font weight under 400 (regular)
    1. Below 400 is really light and hard to read

Creation of a Good Reading Experience

1. Use less than 75 characters per line
    1. 65 - 72 chars
        1. Anything above is hard to read (line is too long)
2. For normal-sized text, use a line height between 1.5 and 2. For big text, go below 1.5 (1.1 or 1.2)
    1. The **smaller** and **longer** the text, the **larger** the line height needs to be!
        1. Big text: 1.2
        2. Smaller text: 1.31
        3. Smaller text: 1.52
    2. Oversized lines give a bad reading experience
3. Decrease letter spacing in headlines, if it looks unnatural (this will come from experience)
    1. -3.5px - natural
    2. 0px - unnatural
4. Experiment with all caps for short titles
    1. Make them small and bold and increase letter-spacing
5. Usually, don't justify text (convention)
    1. It might look unnatural
6. Don't center long text blocks. (makes it harder to read the text)
    1. Small blocks are fine

Implementing Typography

1. Example: Chair design shop
    1. sans-serif
        1. For designers and architects: Inter
            1. fonts.google.com
                1. Styles: font-weights
                    1. Select: 400 (regular)
                    2. Bold: 700
                    3. Medium: 500
                2. Copy `<link>` (a new stylesheet is loaded - comes from a different server)
                    1. Paste before `style.css`
2. Font-size system
    1. 10, 12, 14, ..., 98
        1. type-scale.com
            1. Copy values
    2. Font-sizes should set hierarchy (sizes should differentiate)

Web Design Rules #2: Colors

1. Color theory - complex
    1. We don't need to know this

Choose the right color

1. Make the main color **match your website's personality**: colors convey meaning!
    1. Example: We don't use a pink color on an elegant lawyer's website
    2. Example: We wouldn't use gold for a playful fun startup
    3. Meaning of colors:
        1. **Red**: Draws a lot of attention, and symbolizes power, passion, and excitement
        2. **Orange**: Is less aggressive, and conveys happiness, cheerfulness, and creativity
        3. **Yellow**: Means joy, brightness, intelligence
        4. **Green**: Represents harmony, nature, growth, and health
        5. **Blue**: (Most used) is associated with peace, trustworthiness, and professionalism
            1. Website usually want to convey these
        6. **Purple**: Conveys wealth, wisdom
        7. **Pink**: Represets romance, care, and affection
        8. **Brown**: is associated with nature, durability and comfort
        9. **Black**: symbolizes power, elegance and minimalism, but also grief and sorrow
            1. Difficult color to use
                1. Not the best color for beginners
    4. Use a **good color tone**! Don't choose a random tone or CSS named colors.
        1. Use toolbox:
            1. Open Colors: The colors that are carefully designed for beautiful UIs
            2. Tailwindcss
                1. It is a CSS framework
                    1. We can just use the colors
                        1. Beautifully picked
            3. Flat UI Colors 2
                1. Less shades but popular choice

Establish a Color System

1. You need at least two types of colors in your **color palette**: a **main color** and a **grey color**
    1. **Main color**: greenish blue
    2. **Grey color** (for text): very dark blue
        1. A very dark version of another color is fine
2. With more experience, we can add more colors: **accent (secondary) colors** (use a tool)
    1. **Accent**
        1. It cannot be random color
            1. It must be related to the main color and grey color
                1. Solution: Use Toolbox
3. For diversity, define lighter and darker "versions" (**tints and shades** (especially for grey colors - we might need many for text in different places and different states (button states say)) - Open Color & Tailwindcss come with these, or else use **tint and shade generator**. For **accent color** use a tool like pelleton.com or coolors.com (to generate entire color pallettes)
    1. tints - lighter versions
    2. shades - darker versions

When and How to Use Colors

1. Use your main color to **draw attention** to the most important elements on the page
    1. Example: Main color is used to draw attention to a button (most important element in page - we want user to click)
    2. Exapmle: Main color can be found in the logo
2. Use colors to add **interesting accents** or make **entire components or sections** stand out
    1. Borders
    2. Underline text
    3. Highlighted text
    4. Color behind icons
    5. Colored text
    6. Background color
    7. Gradient (a step further)
3. You can try to use your color strategically in **images and illustrations**
    1. Main color appearing in photos

Colors and Typography

1. On dark colored backgrounds, try to **use a tint of the background** ("lighter version") for text
    1. Looks more natural
2. Text should usually not be completely black. **Lighten it up** it looks heavy and uninviting
3. **Don't make text too light!**
    1. Use a tool to check contrast between text and background colors
        1. For people who have difficulty seeing
            1. Solution: Coolers
                1. Contrast ratio needs to be at least **4.5:1 for normal text** and **3:1 for large text** (18px+)

Implementing Colors

1. Base color:
    1. Green color in pictures can be matched
    2. Focus is also on health
        1. Green works well (nature, growth, health)
    3. Open Color: yeun.github.io/open-color
        1. Teal - Good enough
2. coolers:
    1. Contrast checker

        1. Text color: #ffffff
        2. Background color: #087f5b
        3. Contrast: 5
            1. Good - not perfect
3. Put common styles in one class
4. Put variations in specialized classes

Web Design Rules #3: Images and Illustrations

1. Topics:
    1. How to use images well
    2. When to use images
    3. What images to use

Use Good Images

1. Different types of images:
    1. **product photos**
        1. Illustrates products we are trying to sell on websites
            1. Physical or illustrations
                1. Download and study slides and pdf carefully
    2. **storytelling photos** (most important of the 4)
        1. Convey the message of the website
            1. They don't show the product
                1. They might show someone using the product
                2. They might be something related to the product or message of the site
                    1. Example: Where coaching happens
                    2. Example: Showing someone using a product we want to sell
    3. **illustrations**
        1. More popular these days
        2. They are more abstract way of telling stories
        3. They help to bring out website personality
        4. There are 3D illustrations
    4. **patterns**
        1. They might appear behind sections
        2. They might appear behind images
        3. Purpose: Used for creativity and to add some visual detail to pages
            1. To make the website more interesting
                1. Not too many
2. Use images to support your website's **message and story**
    1. Only use **relevant images!**
3. Prefer **original images** (taken ourselves or a professional photographer takes)
    1. If not possible, use **original-looking** stock images (not generic ones!)
        1. Never use stock photos that are too generic and appear in different places (already seen in many places)
            1. Not fake
            2. Not overproduced
                1. Users look for authenticity than polish
4. Websites for images:
    1. **Unsplash**
    2. **Pexels**
    3. **DrawKit**
    4. **unDraw**

Use Images Well

1. Try to show real people to trigger user's emotions
    1. Not some overproduced models
        1. People connect with real people
2. If necessary, **crop images** to fit your message
3. Experiment **combining** photos, illustrations, and patterns

Handling Text on Images

1. **Method #1**: Darker or brighten image (cmpletely or partially, using a gradient)
2. **Method #2**: Position text into neutral image area
    1. Continuous background color say
        1. Careful when image gets smaller on a mobile screen
3. **Method #3**: Put text in a box
    1. We can add opacity to the box

Some Technial Details

1. To account for **high-res screens**, make image dimensions **2x as big** as their displayed size (twice the number of pixels - 600 x 600 px for 300 x 300 px screen)
    1. Smartphone screen
    2. High-end macbook
    3. Dell XPS
2. Scale factor:
    1. Actual pixels the screen contains / pixels represented on screen
    2. On high-res screens, scale factor is 2x or even 3x (latest iphones), on "normal" screens it's just 1x (1 physical pixel = 1 design pixel)
        1. Each pixel is represented by 2 or 3 pixels
            1. For very crisp and high resolution images
                1. High res screen will show 2 pixels in place of 1 pixel we have specified in CSS
3. **Compress images** for a lower file size and better performance
    1. Tool: Squoosh (96% - 3.18 MB -> 112 KB)
4. When using multiple images side-by-side, make sure they have the **exact same dimentions**

1. At-least same aspect ratio

Web Design Rules #4: Icons

1. Icons can add details and change the look and feel of a page or application
2. Important rules

Use Good Icons

1. Use a **good icon pack**, there are tons of **free** and paid icons packs
    1. Toolbox: (free)
        1. Phospor icons
            1. Very good source
        2. Ionicons
        3. Icons8
    2. Toolbox:
        1. Emojis - good option
2. Use only one icon pack.
    1. **Don't mix** icons from different icon packs
        1. Distracting
3. Use **SVG icons** or **icon fonts**
    1. Don't use bitmap image formats (.jpg, and .png)
        1. SVG images and icon fonts are vector based
            1. They can scale indefinitely
        2. Regular images: JPG, PNG, GIF
            1. Do not scale
                1. Become unsharp
                    1. Problem on high definition screens
                        1. There is a scale factor of 2x
4. Adjust to websie personality! **Roundness**, **weight** and **filled/outlined** depend on typography
    1. Icons are placed along with text usually

When to Use Icons

1. Use icons to **provide visual assistance** to text
2. Use icons for **product feature blocks**
    1. **Feature block**: (to describe features of a product or service)
        1. Icon at the top
            1. May not require reading text
        2. Smaller heading
        3. A description
3. Using icons **associated with actions**, and **label them** (unless no space or icon is 100% clear)
    1. Visual assistance to the text
        1. Makes it easier and faster for users to understand (sometimes we don't have the read the text)
    2. We should still label the icons (many users may not know the meaning of the icons)
        1. Exceptions: Only if there is no space or if icons makes it 100% clear
    3. Be consisten
        1. Don't mix icons only with text etc...
4. Use icons as **bullet points**
    1. Checkmarks
    2. etc.

Use Icons Well

1. To keep icons neutral, **use same color as text** (not draw much attention - focus is on text but icons provide visual assistance). To draw more attention, **use different color**
    1. Different color for all icons
    2. Different colors for different icons
2. Don't confuse your users: icons need to make sense and **fit the text or action we want to convey!**
3. Don't make icons larger than what they **were designed for**. If needed, **enclose them in a shape**
    1. Example: If lines are too thick, they might look unnatural
        1. Solution: Make icon smaller and inclose it in another shape
    2. Example: If icons that use thin lines that can scale we can use them

Implementing Icons

1. Step 1: Choosing an icon pack
    1. heroicons.com
        1. SVG icons
        2. Icons with outline style and solid style
        3. Icons may not be pe rfect fit for the website (boxy style)
        4. Different ways to use them
            1. Download and use
                1. Has some limitations
            2. Copy and use the SVG code
            3. Paste the SVG code directly into the page
2. We use SVG in two situations:
    1. Feature blocks
    2. Visual assistance to text

Web Design Rules #5: Shadows

1. Shadows help users to figure out relationships between parts of our design
2. Shadows can also be used to add intersting visual details

Some Concepts

1. Trends:
    1. Skeuomorphic design
        1. Glossy details
        2. Lots of shadows
    2. Flat design (minimal)
        1. Only essentials
            1. It has some usability problems
    3. Flat design 2.0
        1. Still minimal, but brings back shadows and depth for better usability
2. After an era of 100% flat design, we're now **backto using shadows** in UI design ("Flat design 2.0")
3. **Shadows generate depth (3D)**: The more shadow, the **further away from the interface** the element is
    1. 2px away from interface
        1. Very small shadow
    2. ...
    3. 24px away from interface
        1. A large shadow
4. We can use shadows on **boxes** and **text**

Use Shadows Well

1. You **don't have to use** shadows! Only use them if it makes sense for the **website personality**
    1. Less shadows - serious / elegant
    2. More shadows - playful / fun
2. Use shadows in **small doses**: don't add shadows to every element!
    1. Confusing
        1. Doesn't define any visual hierarchy
            1. It is as good as not having any shadows
                1. Use shadows for elements that you want to stand out from the rest of the elements
3. Go light on shadows, do't make them **too dark**!
    1. Dark shadows look unnatural (they don't exist in real world)

Use Shadows in the Right Situation

1. Use **small shadows** for smaller elements that should stand out (to draw attention) (most important elements that convert users to paid customers for instance)
    1. Action buttons
    2. Small form elements (input for email say)
    3. Images
        1. Purpose: Make them float a little bit (for drawing attention - important for story telling)
2. Use **medium-sizes shadows** for larger areas that should stand out a bit more
    1. To make an entire section stand out from the rest of the page
    2. To make cards stand out of the rest of the interface
        1. Product card - say
            1. Description of a product that user might buy
3. Use **large shadows** for elements that should really float above the interface (entire rest of the interface - shows that the elements are floating above the rest)
    1. Navigations
    2. Pop-up windows
4. Experiment with **changing shadows** on mouse interaction (click and hover)
    1. Normal state doesn't have shadow, then when we hover, we can have medium sized shadow
        1. Once user clicks, we can reduce the shadow to minimum (or zero) giving the effect of pushing the button towards the interface
            1. Gives a natural feel
5. Bonus: Experiment with **glows** (colored shadows)
    1. Shadows don't have to be black, if we give a certain shadow, they make the element glow
        1. Red shadow for red button
            1. Modern effect

Implementing Shadows

1. Shadow to cards: medium size shadow

```
box-shadow: <horizontal-offset-of-shadow-between-box-and-shadow> <vertical-offset> <blur-small-not-blur-large-huge> <optional-scale-up-shadow-0
```

2. `<figure>` **(M)** - a common element used for product card
    1. We can use `<article>`
3. **We usually want the light source to come from the top**
4. **For medium shadow**:

Web Design Rules #6: Border-radius

1. It is pretty important
    1. Border radius has effect on seriousness or playfulness on our design

Use Border-Radius Well

1. Use border-radius to **increase the playfulness** and fun of the design, to make it **less serious**
    1. Less border-radius - serious / elegant
    2. More border-radius - playful / fun
2. Typefaces have a certain roundness: make sure that border-radius **matches that roundness!**
    1. Very round typeface
    2. Round buttons
    3. Icons wrapped inside circles
3. Use border-radius on **buttons**, **images**, **around icons**, **standout sections** and **other elements**

1. Buttons:
    1. Just enough the take away the hard corners
    2. Completely round
    3. Some of the corners round
2. Images
    1. just enough to take away the hard corners
    2. Completely round
    3. Roundy
3. Around icons
    1. Icons completely enclosed in a circle
4. Standout sections
    1. We want certain sections to standout
5. Other elements
    1. Random elements

Implementing Border-radius

1. `border-radius` property on an element

Web Design Rules #7: Whitespace

1. Why Whitespace
    1. The right amount of whitespace makes designs look **clean**, **modern** and **polished**
        1. Content might look cramped & doesn't look clean
        2. Eyes find it difficult to navigate through the page due to the lack of whitespace
            1. If there is whitespace, the design looks like there is space to breathe
    2. Whitespace communicates how different pieces of information are **related to one another**
    3. Whitespace implies **invisible relationships between the elements** of a layout
        1. Intiutive navigation

Where to Use Whitespace

1. Use tons of whitespace **between sections**
    1. Example: 192px, 140px, 160px
        1. We cannot fix one number and use that number
2. Use a lot of whitespace **between groups of elements**
    1. Example: 96px, 152px
    2. Vertically we need more whitespace than horizontally
3. Use whitespace **between elements**
4. Inside **groups of elements**, try to use whitespace **instead of lines**
    1. Space looks more modern

How Much Whitespace

1. **The more some elements** (or groups of elements) **belong together, the closer they should be!**
    1. To distinguish them from other surrounding groups of elements
        1. The law of proximity
    2. If the elements do not belong together, they should be far apart
    3. Example:
        1. Title and text belong together
            1. Less space between them
    4. Example:
        1. Each label clearly belongs to a certain input field
2. Start with **a lot of whitespace**, maybe even too much! Then **remove whitespace** from there (until it looks right)
    1. Too much whitespace looks **detached**, too little looks too **crammed**
3. Match **other design choices**. If you have big text or big icons, you need more whitespace
    1. Big text, lots of whitespace
    2. Small text, and images, less space
4. Try a hard rule, such as using **multiples of 16px** for all spacing
    1. For margins
    2. For paddings
    3. 2px, 4px, 8px, 16px, 24px, ... are exceptions

Web Design Rules #8: Visual Hierarchy

What is Visual Hierarchy?

1. Visual hierarchy is about **establishing which elements** of a design **are the most important ones**
2. Visual hierarchy is about **drawing attention** to these most important elements
    1. Giving prominance to more important elements
3. Visual hierarchy is about **defining a "path" for users**, to **guide** them through the page (in an intuitive and automatic way)
    1. Most important
    2. Less important
    3. Less important
    4. ...
4. We use a combination of **position**, **size**, **colors**, **spacing**, **borders**, and **shadows** to establish a meaningful visual hierarchy between elements/ components.

Visual Hierarchy Fundamentals

1. Position important elements **closer to the top of the page**, where they get more attention
    1. In general, attention flows down the page (and components)
        1. Logo -> Navigation -> Main heading -> Important call to action button -> ...
2. Use images mindfully, as they draw **a lot of attention** (larger images get more attention)
    1. First the attention goes to an image even before the main heading

        1. Smaller images draw lesser attention
    3. Whitespace causes separation, so **use whitespace stretegically** to emphasize elements
        1. Example: A lot of whitespace around the most important parts of a page
            1. Sign up form
            2. Feature blocks

Visual Hierarchy for Text Elements

1. How to make text elements more or less prominant
    1. Using visual hierarchy at a smaller level
    2. For text elements, use **font size**, **font-weight**, **color** (text color and background color), and **whitespace** to convey importance
        1. Title: increase font-size and font-weight
        2. Important elements: increase font-weight
        3. Less important elements: lighten the color
        4. Grouping: Add whitespace
    3. Flow: (Visual hierarchy to guide the user)
        1. Image -> Title -> Button -> Rating -> Price -> Details
        2. Title -> Button -> Bold Text -> Details (de-emphasized, less font-weight)
        3. Title -> Description -> Image -> Name (top-down)
        4. Big text -> smaller text -> smaller text -> ...
2. What text elements to emphasize?
    1. **Titles**
    2. **Sub-titles**
    3. **Links**
    4. **Buttons**
    5. **Data points**
    6. **Icons**
        1. You can also **de-emphasize** less important text, like **labels** or **secondary/additional information**

Visual Hierarchy Between Components

1. Components: Groups of elements or big areas of content
2. How do we do that?
    1. Emphasize an important component using **background color**, **shadow**, or **border** (or multiple)
    2. Try emphasizing some component A over component B by **de-emphasizing component B**
    3. What components to emphasize?
        1. **Testimonials**
        2. **Call-to-Action sections**
            1. Sign-up
        3. **Highlight sections**
            1. In the middle of a content
                1. Different background color than the rest of the page
        4. **Preview cards**
            1. For products or services to sell
        5. **Forms**
        6. **Pricing tables**
            1. Highlight the one we want user to choose
        7. **Important rows/columns in tables**
        8. etc.

Implementing Whitespace and Visual Hierarchy

1. Using whitespace and other techniques

Web Design Rules #9: User Experience (UX)

1. Introduction (UX is a huge field)
    1. How to improve UX of our interface?

What is User Experience (UX)?

1. "Design is not just what it looks like and feels like. Design is **how it works**" - Steve Jobs
    1. **User Interface (UI)** is the visual presentation of a product. It's how the graphical interface looks and feels like
        1. Layout
        2. "Personality"
            1. Feel and vibe we give to our design
        3. Typography, colors, icons, etc.
    2. **User Experience (UX)** is the overall experience the user has while interacting with the product
        1. Does the app feel **logical** and well thought out?
        2. Does the navigation work **intuitively**?
        3. Are users **reaching their goals**? (through the product)
        4. ...

UI and UX Design

1. **UI** is graphical interface
    1. **UI Design** is what makes an interface **beautiful**
2. **UX** is experience with interface
    1. **UX Design** is what makes an interface **useful and functioanl**
3. **UX Design can not exist without UI Design**
    1. A well designed UI is already contributing a lot to a good experience for the user

UX Design Using Principle: Goals

1. A website or application **exists for a reason**: a **user** has a goal for **visiting** it, and a **business** has a goal for **constructing** it
    1. User goals
        1. Use a product to find some info or buy a product
    2. Business goals
        1. To sell something (actual product/ advertisements)
    3. Good UX design **aligns** the **user's goals** with the **business goals**
        1. UX design must fulfill the goals of both the users and the business
            1. Example:
                1. User goal - design websites faster
                2. Business goal - selling design kits for design tools (Sketch or Figma)
                3. Good UX design must fulfill both goals
                    1. A transaction that allows business to sell some design kits so that a user can design a website faster
            2. Example:
                1. Highlighting an option in the product pricing table:
                    1. Helps the **user** decide faster what is the best option
                    2. Helps the **business** maximize revenue
                        1. The highlighted one is usually the most expensive one (maximum profit)
            3. Example:
                1. A popup form to capture user's email address
                    1. It could be an annoying experience for the users - goals are usually not aligned
                        1. popup is not a good UX design usually
                2. Bad UX designs:
                    1. Hiding a button to cancel our subscription or making it almost impossible to find it

UX Rules For Usability

1. Don't design complicated layouts. Don't reinvent the wheel. **Use patterns that users know**
    1. Don't have to original
        1. Creativity is overrated
        2. Everyday designers must use patterns that already exist
            1. Use **well established patterns**
    2. A website exists because users want to reach a goal
        1. If a page looks familiar it will help user to navigate the page faster and therefore they will reach their goal faster
            1. Many websites look almost the same (there is nothing wrong with that)
2. Make your call-to-action the **most prominent element**, and make the **text descriptive**
    1. Example: A button describes exactly what will happen when a user clicks it
        1. This makes sure a user reaches the goal faster and more easily
3. Use **blue text** and **underlined text** only for **links**
4. Animations should have a **purpose** and be **fast**: between 200 and 500 ms
    1. We must not use animations just because we can (might confuse users and can look unprofessional)
    2. If it takes a long time, it might look slow and sluggish
        1. Example: fading in in 2s (sluggish)
5. In forms, align labels and fields in a **single vertical line**, to make the form **easier to scan**
6. Offer users **good feedback** for all actions: form errors, form success, etc. [web apps]
    1. Example: If an email is sent, give feedback to the user that the email was successfully sent. Don't leave the user hanging in confusion
7. Place action buttons where they will **instantiate an effect** (law of locality) [web apps]
    1. Example: In a todo list, an Add task button is at the end of the list. It is the place where a new task will get created
        1. Not in a random place

UX Rules for Website Content

1. Content influences the way a user uses the Website
2. Rules:
    1. Use a **descriptive, keyword-focused headline** on your main page. Don't be vague or fancy!
        1. Keywords should best describe what the business does
            1. Examples: (To help user reach their goals faster)
                1. The AI assistant that grows your money
                2. Automate banner production in minutes
                3. Greenlight makes it easy to leave feedback on any website
                4. THe All-In-One Toolkit for Working Remotely
            2. Bad examples:
                1. Join the solar energy revolution
                2. The way you work is evolving. Is your hiring software?
                3. Meaningful Insights Without the Click of a Button
                4. Is design growing your product?
    2. Only include **relevant information**, efficiently! **Cut out fluff** (irrelevant images, text, illustrations, buttons, ...) and make the content 100% clear (clear purpose)
        1. If we add aditional info to the page, it might compete with other more relevant info
        2. Web-site doesn't have to be super minimal
            1. Don't just add fluff content that doesn't add to overall message we want to convey
    3. Use **simple words**! Avoid technical jargon and "smart-sounding" words
        1. No complicated and smart sounding words
        2. Users may not know the technical jargon
            1. They confuse users
    4. Break up long text with **sub-headings**, **images**, **block-quotes**, **bullet points**, etc.
        1. Makes text more interesting to read
            1. One big wall of text? Users might just skip it
                1. Might have higher user retention

The Website-Personalities-Framework

1. Recap:
    1. 7 website personalities
        1. Once we choose a personality, our design choices will become limited
            1. It will make it easier to make it a lot easier to make a consistent and good looking design on our own
2. What is a website personality?

1. General feeling or vibe that our website wants to transmit to our users
    1. It is also a feeling we want to invoke in our users
    2. Example:
        1. Serious/Elegant: We want use to feel luxury or sophistication or sense of a premium product
3. Steps:
    1. Think about:
        1. How do you want website to **appear to users**?
        2. What **vibe** do you want to transmit?
    2. Look at the list of personalities and choose the personality that best fits the vibe that we want to convey (we can choose multiple personalities as well) (Why does it matter? It will make web-design much easier - design options for
        1. Serious/Elegant
            1. Doesn't use shadows
            2. Doesn't use border-radius
        2. Minimalist/Simple
        3. Plain/Neutral
        4. Bold/Confident
        5. Calm/Peaceful
        6. Startup/Upbeat
            1. Uses shadows
            2. Uses border-radius
        7. Playful/Fun
    3. Apply **personality traits** to each **design ingredient**
        1. Personality Traits:
            1. Typography
            2. Colors
            3. Images
            4. Icons
            5. Shadows
            6. Border-radius
            7. Layout
        2. Personality 01 - Serious/Elegant (download slides and study the personalities)
            1. **Overview**: Design for luxury and elegance, based on **thin serif typefaces**, **golden or pastel colors**, and big high-quality images
            2. **Industries**: **Real estate**, **high fashion**, **jewelry**, **luxury products** or **services**
            3. Ingrediants:
                1. **Typography**: Serif typefaces (especially in headings), light font weight, small body font size
                2. **Colors**: (luxury or high quality products)
                    1. Gold
                    2. Pastel colors
                    3. Black
                    4. Dark blue or grey
                3. **Images**
                    1. Big
                    2. High-quality images (professional photography)
                        1. To feature elegant and expensive products
                4. **Icons**
                    1. Usually no icons
                        1. But thin icons and lines may be used
                5. **Shadows**
                    1. Usually no shadows
                6. **Border-radius**
                    1. Usually no border-radius
                7. **Layout**
                    1. A creative and experimental layout is quite common
        3. Personality 02 - Minimalist / Simple
            1. **Overview**: Focuses on the essential text content, using **small or medium-sized sans-serif black text**, **lines**, and **few images and icons**
            2. **Industries**:
                1. Fashion
                2. Portolios
                3. Minimalism companies
                4. Software Startups
            3. Ingrediants:
                1. **Typography**
                    1. Boxy/sqaured sans-serif typefaces
                    2. Small body font size
                2. **Colors**
                    1. Usually black or grey
                    2. On pure white background
                    3. Usually just one color throughout the design
                3. **Images**
                    1. Few images
                        1. Used to add some color to the design
                    2. Usually no illustrations (especially no 3D illustrations)
                        1. If illustrations are used, they are just black
                4. **Icons**
                    1. Usually no icons
                        1. But small simple black icons may be used
                5. **Shadows**
                    1. Usually no shadows
                6. **Border-Radius**
                    1. Usually no border-radius
                7. **Layout**
                    1. Simple layout
                    2. A narrow one-column layout is quite common
        4. Personality 03 - Plain / Neutral (no much visual impact. Almost boring)
            1. **Overview**
                1. Design that gets out of the way by using **very neutral and small typefaces**, and a boxy, structured, and condensed layout
            2. **Industries**

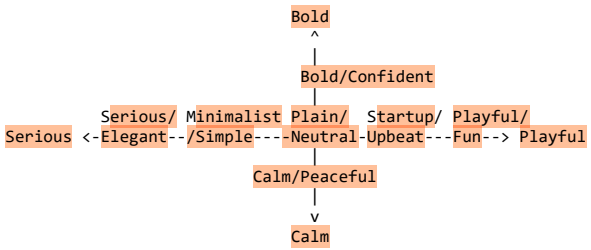
        1. Well-established corporations
        2. Companies that don't want to make an impact through design
    3. Ingrediants:
        1. **Typography**
            1. Neutral-looking sans-serif typefaces are used, and text is usually small and doesn't have visual impact
        2. **Colors**
            1. Safe colors are employed, nothing too bright or too washed-out, Blues (conveys trust) and blacks are common
        3. **Images**
            1. Images are frequently used, but usually in a small format
                1. They don't take a center stage like they do in other website personalities
        4. **Icons**
            1. Usually no icons, but simple icons may be used
        5. **Shadows**
            1. Usually no shadows
        6. **Border-Radius**
            1. Usually no border-radius
        7. **Layout**
            1. Structured and condensed layout, with lots of boxes and rows
5. Personality 04 - Bold/Confident
    1. **Overview**: Design that makes an impact, by featuring **big and bold typography**, paired with confident use of **big colored blocks**
    2. **Industries**:
        1. Digital agencies
        2. Software startups
        3. Travel
        4. "Strong" companies
            1. Companies that want to transmit a strong image
                1. That they are confident in themsleves and are ready to make some bold moves
    3. Ingredients
        1. **Typography**
            1. Boxy/squared sans-serif typefaces
            2. Big and bold typography
                1. Especially headings
            3. Uppercase headings are common
        2. **Colors**
            1. Usually multiple bright colors
            2. Big color blocks/sections are used to draw attention
        3. **Images**
            1. Logs of big images are usually displayed
        4. **Icons**
            1. Usually no icons
        5. **Shadows**
            1. Usually no shadows
        6. **Border-Radius**
            1. Usually no border-radius
        7. **Layout**
            1. All kinds of layouts
            2. No particular tendencies
6. Personality 05 - Calm/Peaceful (complete opposite of boldness or confident personality) (shows that the company cares about the consumers, text is all about the consumer - helping and caring about them)
    1. **Overview**
        1. For products and services that care about the consumer, which is transmitted by **calming pastel colors** and **soft serif headings**
    2. **Industries**
        1. Healthcare
        2. All products with focus on consumer well-being
    3. Ingrediants:
        1. **Typography**
            1. Soft serif typefaces frequently used for headings
                1. But sans-serif headings might be used too
                    1. Example: for software products
        2. **Colors**
            1. Pastel/Washed-out colors: light oranges, yellows, brows, greens, blues
        3. **Images**
            1. Images and illustrations are usual
                1. Matching calm color palette
                2. Usually have people
        4. **Icons**
            1. Icons are quite frequent
        5. **Shadows**
            1. Usually no shadows
                1. But might be used sparingly
        6. **Border-Radius**
            1. Some border-radius is usual
        7. **Layout**
            1. All kinds of layouts
                1. No particular tendencies
                    1. No strange or creative layouts are used
7. Personality 06 - Startup/Upbeat (Technology websites)
    1. **Overview**: Widely used in startups (software), featuring **medium-sized** sans-seriff typefaces, **light-grey backgrounds**, and rounded elements (Stripe website is a good example)
    2. **Industries**:
        1. Software startups
        2. Other modern-looking companies
    3. Ingrediants:
        1. **Typography**
            1. Medium-sized headings (not too large)
            2. Usually one sans-serif typefaces in whole design
            3. Tendency for lighter text colors

2. **Colors**
    1. Blues
    2. Greens and purples are widely used
    3. Lots of light backgrounds (mainly gray)
    4. Gradients are also common (popular)
3. **Images**
    1. Images or illustrations are always used
    2. 3D illustrations are modern
    3. Sometimes patterns and shapes add visual details
4. **Icons**
    1. Icons are very frequent
5. **Shadows**
    1. Subtle shadows are frequent
    2. Glows are becoming modern
6. **Border-Radius**
    1. Border-radius is very common
7. **Layout**
    1. Rows of cards (or rows of product features) and Z-patterns are usual, as well as animations

8. Personality 07 - Playful/Fun
    1. **Overview**
        1. **Colorful and round** designs
        2. Fueled by **creative elements** like hand-drawn icons or illustrations
        3. Animations
        4. Fun language
    2. **Industries**
        1. Child products
        2. Animal products
        3. Food
    3. Ingrediants
        1. **Typography**
            1. Round and creative (e.g. handwritten) sans-serif typefaces are frequent
            2. Centered text is more common
        2. **Colors**
            1. Multiple colors are frequently used to design a colorful layout
            2. All over in backgrounds, text, and images
                1. To give a playful and fun vibe
        3. **Images**
            1. Images
            2. Hand-drawn (or 3D) illustrations
            3. Geometric shapes and patterns are all very frequently used
        4. **Icons**
            1. Icons are very frequent
            2. Many times in a hand-drawn style
        5. **Shadows**
            1. Subtle shadows are quite common
                1. But not always used
        6. **Border-Radius**
            1. Border-radius is very common
        7. **Layout**
            1. All kinds of layouts
                1. No particular tendencies

Advanced: Combining Playfulness and Boldness

1. Placing the 7 personalities into 4 quadrants:



    1. Playful:
        1. Colorful
        2. Rounded corners, typography and icons
        3. Shadows
        4. Illustrations
    2. Bold:
        1. Boxy/squared sans-serif typefaces
        2. Big and bold typography
        3. Bright/flashy colors
    3. Calm:
        1. Headings using soft serif typefaces
        2. Pastel/washed-out colors
        3. Illustrations
    4. Hybrid:
        1. Boldness/Confident personality can get injected into one of the 5 personalities
            1. Startup/Upbeat can get pushed towards Bold/Confident personality
                1. Example:
                    1. Irregular round design elements
                    2. Hand-drawn icons and patterns
            2. Startup/Upbeat can get pushed towards Balm/Peaceful personality

            1. Example:
                1. **Headings using soft serif typefaces**
                2. **Illustrations in calming pastel colors**
        2. Calmness/Peacuful personality can get injected into one of the 5 personalities

The Missing Piece: Steal Like An Artist!

1. Getting inspirations - for coming up with designs on our own
2. **Copy what we like from other great designs** (stealing like an artist)

The Missing Piece

1. Ingredients
    1. Website Personalities
    2. Design Ingredients
    3. Design Guidelines & Rules
    4. **Getting inspired and stealing like an artist**
        1. **Not** completely **copying** a design instead, it's about **taking good parts and adapting them** to our needs while using all rules and guidelines
2. Aggregator sites:
    1. land-book.com
        1. Example: Designing a website for startup
            1. Startup and upbeat personality
            2. Apply design ingredients for the personality
            3. Inspiration
                1. Select a page
                    1. Endless feed for designs
    2. onepagelove.com
    3. awwwards.com - more creative and superior (needs more experience to choose)
        1. Gives awards for creativity and innovation
            1. Collections
                1. Websites
    4. screenlane.com
        1. For individual components
3. **If you see many designs, it becomes easy to design**
    1. Take inspiration
        1. Take 1/2 hour to look in the websites every month
        2. We will grasp different patterns
            1. Pattern: Repititive components
                1. There are common patterns that most pages use

Section 6: Components and Layout Patterns

Section Intro

1. Common website components
2. Common layout patterns
    1. Sliders
    2. Testimonials
    3. Prizing tables
    4. Navigations
    5. Z-Pattern
    6. ...
3. We can use the common patterns and components to build our own pages with confidence
4. Building some common components

Web Design Rules #10 - Part 1: Elements and Components

1. One important ingredient (missing)
    1. Designing components and layouts
        1. We need general guidelines
        2. We need a lot of inspiration

From Elements to Webpage

1. How is a webpage designed from scratch?
    1. We can use well established patterns (we build websites in the say way)
    2. Steps:
        1. Step 1: We start with common small elements:
            1. Paragraphs
            2. Images
            3. Buttons
        2. Step 2: We then assemble the simple elements into common components
            1. Feature row
            2. Feature card
            3. Pricing table
            4. Tabbed component
        3. Step 3: We build layout using common layout patterns
        4. Step 4: We take smaller layouts and assemble them into the final webpage
2. We use **common elements** and **components** to convey our website's information
3. We combine components into layouts using **common layout patterns**
    1. Repeating certain components
4. We assemble different **layout areas** into a complete, final page

Gallery Index: Elements, Sections, Patterns

1. Common:
    1. Elements
        1. Text
        2. Buttons
        3. Images
        4. Input elements
        5. Tags
    2. Components (elements are combined into components)
        1. Breadcrumbs
        2. Pagination
        3. Alert and status bars
        4. Statistics
        5. Gallery
        6. Feature box
        7. Preview and profile cards
        8. Accordion
        9. Tabs
        10. Carousel
        11. Customer testimonials
        12. Customer logos
        13. Featured-in logos
        14. Steps
        15. Forms
        16. Tables
        17. Pricing tables
        18. Modal windows
        19. ...
    3. Section Components
        1. Navigation
        2. Hero Section
        3. Footer
        4. Call-to-action section
        5. Feature row
    4. Layout Patterns
        1. Row of boxes or cards
        2. Grid of boxes or cards
        3. Z-pattern
        4. F-pattern
        5. Single-column
        6. Sidebar
        7. Multi-column/magazine
        8. Asymmetry/Experimental

Text

1. Inspiration: Ways to start different website sections
    1. Small and short super-descriptive title before the title

Buttons

1. Inspiration: Different types of buttons
    1. Single buttons
    2. Combination of buttons
        1. Primary action button
        2. Secondary action button (less emphasis)

Images

1. Inspiration: Check the slides

Input Elements

1. Single input elements: (check the slides)
    1. One field with email address
    2. Simple dropdown menu
    3. Simple value pickers
    4. ...

Tags

1. Simple elements to construct visual representation of categories
2. Inspiration: check the slides

Breadcrumbs

1. Used to help user navigate through complex sites (check the slides)
    1. Use-case: We use if the website has a deep level of page organization
        1. Simple landing pages may not need

Pagination

1. If we have many results in a list, we put them in many pages & we put some buttons to navigate between the pages
2. Inspiration: Check the slides

Alert and Status Bars

1. Small components that appear at the very top of the page with some announcements (usually)
2. Small alerts for web-applications
    1. Favorited items
    2. Account has been connected
    3. ...
3. Use-cases: To show that something new has happened or some action has happened recently

Statistics

1. Some numbers we want to show to the user about a product or service
2. Very powerful in convincing users that our product is really good for them
3. Inspiration: Check the slides

Gallary

1. Different ways we can do them
    1. Structured
    2. Unstructured ways (in some situations)
        1. Depends on website personality
2. Inspiration: Check the slides

Feature Box

1. It groups some information about something about a product.
2. Things in common:
    1. They use an icon / image / link
    2. They use bigger text for headline
    3. They use some description
3. We can combine them into rows and grids
4. Inspiration: check the slides

Preview and Profile Cards

1. Similar to feature boxes
    1. But they contain some data points for something they want to preview
        1. Product
        2. Profile
        3. ...
    2. We can usually click on the card and it will lead to a different page (usually)
    3. Profile cards
2. Inspiration: Check the slides

Accordion

1. It is a way to hide information and only display once a user clicks on one of the parts
    1. Use-cases: Most commonly used in frequently asked questions
        1. It might be used in other parts of the website where we want to hide some content in the beginning in order to not clutter up the entire page
2. Inspiration: Check the slides

Tabs

1. Very common
2. Used to hide info and show the info once a tab is clicked
3. Use-cases:
    1. For product features
    2. For bigger areas of content we want to hide and show selectively
4. Types:
    1. Horizontal tabs
    2. Vertical tabs
5. Inspiration: Check the slides

Carousel

1. Another way to have multiple pieces of content in the same place or area (aka slider)
    1. User doesn't click on tabs
    2. User clicks on buttons that will move them to the right or to the left which will change the content
2. Common elements:
    1. Dots in the bottom
        1. Shows how many slides
    2. Arrows on the left and the right sides

Customer Testimonials

1. We ask customers for their opinion on the product or service
    1. Most important piece of infomration on modern websites (especially when selling some products or services)
        1. Must not leave out this section if we are building a site that sells products or services
2. Inspiration: check the slides
3. Types:
    1. Wall of tweets
        1. Take some tweets and display them as testimonials
            1. Very powerful way
                1. They show they are true (others may be fake, and user is not sure)

    2. We can also use carousels to display testimonials
4. Inspiration: check the slides

Customer Logos

1. Very important
    1. Especially for marketing websites
2. Take the biggest and most important and most well-known customers and display them in one section
    1. It helps build trust in the brand or in the company the website belongs to
3. As soon as we have 2 or 3 customers, start displaying this

Featured-In Logos

1. Use-Case: Used to build trust with customers
    1. Social proof
    2. Difference between this and customer logos
        1. These are from places where the website or products are featured in
            1. Similar to newspaper or blog

Steps

1. Very effective to show a customer how a product or service works
    1. We can have numbers
    2. It takes the customer through starting the use of the product
    3. It is one of the ways to show how the website or product works
2. Types:
    1. Horizontal
    2. Vertical

Forms

1. Crucial component
2. Types:
    1. Labels can be inside or outside
    2. Login or sign-up forms
        1. Different types
    3. Bigger forms
    4. One-line forms

Tables

1. Pretty important to display tabular data
2. Types: Look at slides
    1. Can also be cards arranged as table

Pricing Tables

1. Very important
2. Very common on marketing pages
    1. For charging money
3. Format:
    1. A column for each pricing plan
        1. Each column has
            1. Title
            2. Description
            3. Price
            4. Features included
            5. Button to sign-up for the plan
        2. Most popular one is highlighted
    2. Creative types - check the slides

Modal Windows

1. Functional modal windows
    1. Why? They are important for the application or the site itself
        1. For displaying data on the site
        2. To allow a user to sign-in or sign-up for the service
2. Marketing modal windows
    1. They want to have mail and want to give something in return
    2. They are still valid

Building an Accordion Component - Part 1

1. Starter files:
    1. 06-Components

Switching Flex-Direction to Column

1. Example:

```
.accordion {
    display: flex;
    gap: 24px;
    flex-direction: column; /* Main axis becomes vertical - top to bottom (from left to right) */
}
```

1. Cross axis - left-to-right (horizontally)
2. `align-items` aligns items **horizontally**, no longer vertically
3. `justify-content` aligns items **vertically**, no longer horizontally
4. `gap` acts like `margin-bottom`, no longer like `margin-right`

Building an Accordion Component - Part 2

1. To copy an element, we can collapse and copy
2. Trick to keep only one item open while all the other items are closed:
    1. By default make all the items as closed
    2. Add a special class that will open one of the items

Building a Carousel Component - Part 1

1. We can style elements and then set the layout for small components

Vertical Centering with Absolute Position and Tranform

1. Example:

```
.btn--left {
    position: absolute;
    top: 50%; /* moves element by exactly 50% of the height of carousel (parent) */
    transform: translate(0, -50%); /* **(M)**, Moves the element 50% of its height to the top */
}

.carousel {
    position: relative;
}
```

Building a Carousel Component - Part 2

Building a Table Component - Part 1

1. New HTML file: 03-table.html
    1. Tables are no longer used in web-development - especially for building layouts
        1. Use-cases: For dataset that can be displayed using a table (most effective)
            1. Semantically, if we need a table, it is better to use a table instead of a grid.
2. Usually in a table, the first row is different
    1. We can wrap the first row inside `<thead>` **(M)** element and the rest of the rows in `<tbody>` **(M)**
    2. `<th>` - table head (in place of `<td>`)

Building a Table Component - Part 2

1. Styling:
    1. We don't format the `<thead>` or `<tr>` but `<th>` and `<td>`
    2. Selecting any descendent:

```
thead th {

}
```

CHALLENGE #1: Building a Pagination Component

1. `margin: 0 auto` works only if we define `width` for the inner element

Web Design Rules #10 - Part 2: Layout Patterns

1. Bigger components - section components & layout patterns

Navigation

1. If user clicks or hovers over a menu item, a sub-menu reveals itself
    1. sub-menu - small box that is over the page
        1. Used when there are multiple sub-pages
    2. Example: Panel occupying an entire width of a page
        1. If we have many pages
    3. Exmaple: Parts of website are described by short sentences
        1. A small summary
    4. Example: Sidebar
    5. Example: Product images
    6. Example: Overlays (rest of the page)
        1. Nice effect
2. Secondary navigation:
    1. Second navigation bar
        1. Shows the user all the other pages that are inside the current main navigation item
            1. Sub-pages in a second menu

Hero Section

1. First section of the page
    1. Content:
        1. Description of page
        2. Main heading

            3. Buttons
            4. Images
    2. Trend:
        1. Trend 1: Text on one side, image on another side
            1. Most common
            2. Format:
                1. Buttons
                2. Headlines
                3. Text description of product or service
        2. Trend 2: Background image with text on top
            1. Used to be popular
            2. It is getting replaced by type 1
        3. Trend 3: First text, then image below
        4. Trend 4: Hybrids
    3. Check the slides

Footer

1. Trend:
    1. Includes complete sitemap
        1. If website is very big, it doesn't make sense
            1. Secondary or tertiary links can be shown
                1. Company info
                2. Privacy policies
                3. ...
    2. Include social media links
    3. Include a submit form
        1. Sign-up for newsletter
        2. ...
    4. We can have simple footer
        1. Minimalistic

Call-to-Action Section

1. They are close to the end of the page
    1. After presenting all the info, we want user to take some action
        1. Subscription to newsletter
        2. Trying a free trial
        3. Contact company
        4. ...
2. It is common for them to stand-out
    1. Using visual hierarchy
        1. Using a different color
        2. Using background image
3. Example: Marketing page
    1. Call to action at the end of the page
4. **Check the slides**
    1. Adding social proof makes it more effective
        1. Customer logos
        2. ...
    2. Re-inforce product features
        1. One-time purchase
        2. 30-day money-back guarantee
5. Ask user to contact
    1. Email
    2. Phone No
    3. Link to contact form

Feature Row

1. Small sections of a page that usually describe some product or service
    1. Multiple rows are combined using a pattern
2. It is usually simple
    1. One side has text
        1. Small heading
        2. Description
        3. Button that leads to a page that explains the feature a little bit better
    2. The second side has an image
3. Check the slides
4. Effectiveness:
    1. Testimonial related to the feature displayed in the row
        1. Very powerful way to sell the product or service
    2. **Using a small uppercase title just before the main heading**
        1. Trend

Row of Boxes/ Cards

1. Layout pattern:
    1. A specific way of repeating a component multiple times
        1. Simple pattern: row of boxes or cards
            1. A very common pattern

Grid of Boxes / Cards

1. Common in online shopping sites

2. To display features on a website

ASIDE: Nesting Patterns in Components

1. Inside each component, we have one of the two patterns
    1. Row of cards
    2. Grid of cards
2. **We can have nested patterns**

Z-Pattern

1. A combination of feature rows
2. A way of repeating similar feature rows but with alternating configurations
    1. Image first, text second
    2. Text first, Image second
    3. Image first, text second
    4. ...
3. Why?
    1. Eyes have a tendency of scanning opposing rows
        1. Image first, text next, text in second row next, image in second row etc.
4. Most of the time, 3 rows are used
    1. We can have less or more

F-Pattern

1. Repeating feature rows
    1. Configuration does not alternate
        1. Image on the left, content on the right
        2. Image on the left, content on the right
        3. ...
    2. We can have inverted F pattern
        1. Content, Image
        2. Content, Image
        3. ...

Single Column

1. Mostly used on responsive websites for mobile phones or on **simple websites**, **simple blogposts (where we don't want distracting sidebars)**

Sidebar

1. Use-case:
    1. Blogs
    2. Options for web-application
    3. Table of contents
    4. Summary

Multi-Column / Magazine

1. Combining different columns of different width into a layout (newspaper or magazine layout)

Asymmetry / Experimental

1. For creativity and stand-out designs
    1. For a different impact to users
        1. Possible with CSS grid
2. A small section of a website can have this layout

Building a Hero Section - Part 1

1. Viewport height:

```
height: 100vh; /* 100% of viewport-height */
```

2. Background image:

```
background-image: linear-gradient(rgba(34, 34, 34, 0.6), rgba(34, 34, 34, 0.6)), url(/hero.jpg);
```

    1. `url` - relative to the current page

Building a Hero Section - Part 2

1. For layout, think about boxes and how to lay the boxes out in the layout
    1. Logic is used to push them around

Building a Web Application Layout - Part 1

1. CSS grid is used for the layout

Building a Web Application Layout - Part 2

2. Scrolling:

```
overflow: scroll;
```

Section 7: Omnifood Project - Setup and Desktop Version

Section Intro

1. A full project with HTML, and CSS learnt so far and more.

The 7 Steps to a Great Website

1. Designing and building a website
    1. It is not just writing code
    2. It requires careful thinking and planning

The Process Behind Building a Website

1. **Define** (foundation - must do)
    1. Define the project
        1. Define **WHO the website is for**. Is it for yourself? (What)
            1. For a client of your agency
            2. For your freelancing business
        2. Define **WHAT the website is for**. In other words, define **business and user goals** or your website project (see lecture on UX)
            1. Purpose: (most common goals below)
                1. To provide info?
                2. To sell some product?
                3. To entertain a user?
            2. Define business goals
            3. Define user goals
            4. This step can be simple and quick
            5. Example: Premium dog-food
                1. Business goal: Selling premium dog food
                2. User goal: Finding high-quality dog food for good price
        3. Define a **target audience**. Be really specific if possible and if it makes sense for your website (this can come form your client)
            1. We need to do this if audience is not well-defined
            2. Exmaple: "Women, 20 to 40 years old, living in Europe, earning over 2000 Euros/month, with a passion for dogs"
                1. We may not be able to be so specific
                    1. It is better to have a precise idea about for whom we are building the website
                        1. **The better we know who the website is for and what we are trying to achieve with it, the more success the project will have**
                            1. Marketing - deep study:
                                1. Competition research
                                2. User personas
2. **Plan**
    1. Plan the project
        1. It must be based on
            1. Business goals
            2. User goals
            3. Target audience
        2. Thinking about the website
        3. Steps:
            1. Plan and gather **website content**: copy (text), images, videos etc. (this guides the design of the website)
                1. Do not come up with a pretty design and then fill the content
            2. Content is usually **provided by the client**, but you also can help them produce and find some content (simply finding free images is easiest, but if they want copy, charge them extra)
                1. Big company: They do it
                2. Small company: They don't have the content yet
                    1. We need to write it ourselves
                        1. Charge extra in this case
                            1. finding images is okay
                            2. Writing content can be charged
                    2. Hire a copywriter
            3. For bigger sites, plan out the **sitemap**: what pages the site needs, and how they are related to one another (content hierarchy)
            4. Based on the content, plan what **sections** each page needs in order to convey the content's message, and in which order
                1. Must convey the message to customer as good as possible
            5. Define the **website personality** (see web design section)
                1. Use content, sitemap etc. to come up with a website personality
3. **Sketch**
    1. Layout
    2. Initial sketches
    3. Sketch layout and component ideas:
        1. Think about what **components** you need, and how you can use them in **layout patterns** (Get inspiration in web design section)
            1. Content should guide the design
        2. **Get ideas out of your head**: sketch them with **pen and paper** or with some design software (e.g. Figma)
            1. Should be very simple
            2. Very low fidelity without any details
                1. No text
                2. Just boxes for images and text content
        3. This is an **iterative process**: experiment with different components and layouts, until you arrive at a first good solution
            1. Experiment with different components and layouts
            2. While coding we can still change things
        4. You don't need to sketch everything, and **don't make it perfect**. At some point, you're ready to jump into HTML and CSS
            1. We can jumpt between sketches and code
            2. We can try different ideas and experiment around
4. **Design and Build**
    1. Design and Build Website:
        1. Use decisisions, content and sketches from step 1, 2, and 3 to **design and build the website with HTML and CSS** ("designing in the browser")
            1. Do the actual design in code

    2. You already have the **layout** and **components** that you selected in Step 3. In this step, you need to design the actual **visual styles**
    3. Build the design based on selected **website personality**, the **design guidelines** shown, and **inspiration** (See web design section)
    4. use the **client's branding** (if it exists already) for design decisions whenever possible: **colors**, **typography**, **icons**, etc.
        1. Web-design must match the client-branding
            1. Client has already made some of the decisions
5. **Test and Optimize**
    1. Make sure website works well in **all major browser** (Chrome, Firefox, Safari, Edge, maybe even old IE (most code may not work))
    2. Test the website on **actual mobile devices**, not just in DevTools
    3. Optimize all **images**, in terms of dimensions and file size (See lecture on images)
        1. Large images are bad user-experience
    4. Fix simple **accessibility** problems (e.g. color contrast issues)
    5. Run the **Lighthouse** performance test in Chrome DevTools and try to fix reported issues
    6. Think about **Search Engine Optimization** (SEO)
        1. A whole course is required
6. **Launch**
    1. Launch the Masterpiece
        1. Once all work is done, everything is perfect, and you got approval from your client (or yourself), it's time to **share your masterpiece with the world!**
        2. Upload your website files to a **hosting platform**. There are countless platforms, we will use one with a free plan (Netlify)
        3. Choose and buy a great **domain name**, one that represents the brand well, is memorable (for users) and easy to write (and must represent the brand of the company the website is for)
7. **Maintain and Update**
    1. After going live
    2. Maintain and Keep Updating Website
        1. Launching is not the end...
        2. Keep the website content **updated over time**. If you're working with a client, you can create a monthly maintenance contract (recurring revenue)
        3. Install **analytics software** (e.g. Google Analytics or Fathom (privacy focused)) to get statistics about website users. This may **inform future changes** in the site structure and content
            1. Used for fine-tuning of the website
                1. We can make it better and better over time
        4. A **blog** that is updated regularly is a good way to keep users coming back, and is also good for SEO
            1. If we use it with the right content
            2. It must be updated regularly

Defining and Planning the Project (Steps 1 and 2)

1. Your first real-world project

    1. Omnifood: fictional startup
        1. Use AI to make and deliver custom healthy meal plans
        2. A new modern, clean website
            1. That will make them look like professional trustworthy company
        3. **Your first "job"!**
            1. Hired by omnifood
            2. Marketing website
        4. **You were hired to design and build a website** for a fictional company called Omnifood
        5. Omnifood is startup that uses AI to construct and deliver custom healthy meal plans
        6. They provided us with all the content for the website (`content.md`)

2. Start files: 07-Omnifood-Desktop

    1. content
        1. img
        2. content.md
            1. Brand color
            2. ...

Step 1: Define the Project

1. Define WHO the website is for
    1. For a client
2. Define WHAT the website is for
    1. Business goal: Selling monthly food subscription
    2. User goal: Eating well effortlessly, without spending a lot of time and money
3. Define target audience
    1. Busy people who like technology (geeky), are interested in a healthy diet, and have a well-paying job (since subscription is not that cheap)

Step 2: Plan the Project

1. Plan and gather website content (done)
2. Plan out the sitemap
    1. We will just buil a **one-page marketing website** (oftentime called a landing page), so no sitemap
3. Deine website personality
    1. Based on the tech-centered target audience, as well as the actual product being sold, we will use the **startup/upbeat** personality. We might add some elements of the **calm/peaceful** personality, since the product is all about consumer well-being as well.
4. Plan page sections
    1. Content guides the design
    2. Hero section:
        1. Branding
        2. Headline
        3. Summary
    3. Features
    4. ...
    5. List of sections:
        1. Logo + Navigation
            1. To lead to different sections of the page

2. Hero
3. Featured in
4. How it works
5. Meals (and list of diets)
6. Testimonials + gallery (together makes it powerful message)
7. Pricing + features
8. CTA
9. Footer

Sketching Initial Layout Ideas (Step 3)

First Ideas and Sketch

1. We don't arrive at a perfect solution immediately, but usually after experimenting around
2. We don't have to think about the entire page at the same time
3. Components for some of the sections
    1. Hero - an example or inspiration
        1. Hero image on the right side
        2. Headline
        3. Summary
    2. Featured in - an example
        1. Logos one besides the other
    3. How it works - an example
        1. Z-Pattern with cards
    4. Meals
        1. Card - for each meal
            1. Image
            2. Data
            3. Icons
    5. Testimonials (one side), Gallary (other side)
        1. Testimonials: 2x2 grid
        2. Gallary: 3x4 images

First Design and Development Steps (Step 4)

1. Responsive design needs to be cared about from the beginning of a project. (adapting to all screen-sizes)

Responsive Design Principles

1. We need to think about it from the very beginning

What is Responsive Design?

1. Responsive Design
    1. Design technique to make a webpage adjust its layout and visual style to **any possible screen size** (window or viewport size)
        1. Technique: Adjusting layout of a website to any possible screen-size
            1. screen-size: Just the viewport size
    2. In practice, this means that responsive design makes websites usable on all devices, such as **desktop computers, tablets, and mobile phones**.
    3. It's a set of practices, **not a separate technology**. It's all just CSS!

Responsive Desig Ingredients

1. Ingredients:
    1. Fluid Layouts
        1. To allow webpage to adapt to the **current viewport** width (or even height)
            1. Flex-box & CSS grid are fluid by default
        2. Use % (or vh / vw) unit instead of px for elements that **should adapt to viewport (usually layout)**
            1. For creation of fluid layout
                1. For things that must adapt to the viewport
        3. Use `max-width` instead of `width`
    2. Responsive Units
        1. Use `rem` unit instead of `px` for most lengths
        2. To make it easy to **scale the entire layout down** (or up) automatically
        3. **Helpful trick**: setting `1rem` to `10px` for easy calculations
    3. Flexible Images
        1. By default, images **don't scale automatically** as we change the viewport, so we need to fix that
            1. They need to adapt nicely for viewport size
        2. Always use % for image dimensions, together with the `max-width` property
    4. Media Queries
        1. Bring responsive sites to life!
            1. Brings other ingredients together
            2. Responsive web-designs will not work without media-queries
        2. To change CSS styles on **certain viewport widths** (called breakpoints)
            1. Allows developers to construct different versions of the website for different devices (for different widths)
            2. Pre-requisite: Fluid layout
                1. We usually write media queries only after building a certain page or certain component
    5. Works just as fine as mobile-first approach

Desktop-First vs. Mobile-First Development

1. Desktip-First:
    1. Procedure:
        1. Start writing CSS for the desktop: **large screen**
        2. Then, media qeries **shrink design** to smaller screens

        1. Traditional way
        2. Easier way to learn (we will use this)
2. Mobile-first:
    1. Procedure:
        1. Start writing CSS for mobile devices: **small screen**
        2. Then, media queries **expand design** to a large screen
        3. Forces us to reduce websites and apps to the **absolute essentials**
    2. Why?
        1. We need to think about mobile experience for our users
            1. We can do that by reducing our website to absolute essentials and stripping away everything that is not entirely necessary
            2. This is more modern way of building websites

How rem and max-width Works

1. Behaviour:
    1. When there is no more additional space to fit the `width` of the element, element should have the width of the parent container

```
max-width: 1000px;
/* When screen size is larger than 1000px, the width is 1000px but if the screen size is less than 1000px, the width of the element is the
```

2. `rem` - root element's font-size **(M)**
    1. Root of a document: `<html>` element (parent of all the others)
        1. If we don't define any font-size for the HTML element, `1rem` = default browser font-size (16px)
            1. Unless user changes it
    2. Why are we defining lenths in terms of font-size?
        1. **If we change the font-size, it will automatically change all the lengths in the CSS with the rem unit**

Building the Hero - Part 1

1. Hero section layout
    1. Customer photos with relevant data
2. Split editor:
    1. Click top right icon
3. `<section>` - more than one should exist inside a container

Building the Hero - Part 2

1. Helper class:

```
.margin-right-sm {
    margin-right: 1.6rem !important;
    /* this must have the highest priority */
}
```

Building the Hero - Part 3

1. Google Fonts: Inter
    1. Inter is perfect for just startup
        1. We need to use rounder face for calmness (inter is boxy)
        2. Solution: Rubik - rounder corners
            1. This needs experimentation

Building the Header

1. Simple:
    1. Logo on the left side
    2. Simple navigation on the right side
2. We don't want to use a fixed length container
3. Style.css:
    1. Put general and repeatable components at the beginning
    2. Put divisions between sections

Building the Navigation

Setting Up a Reusable Grid

1. We can re-use over and over again
2. How it works section
3. Hero section is wider than other sections (130rem as compared to 120rem)

Building the How-It-Works Section - Part 1

1. We can use re-usable classes

Building the How-It-Works Section - Part 2

1. Generic class for centering items in grid vertically:

```
.grid--center-v {
    align-items: center;
}
```

2. z-index: It is confusing

1. Elements having higher value of z-index will appear on top
    1. To push an element behind:

```
z-index: -1;
```

3. Circles behind image:

```
.step-img-box::before,
.step-img-box::after {
  content: "";
  display: block;
  border-radius: 50%;

  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
  /* we want the element to occupy the space of the image so flexbox will not work */
}

.step-img-box::before {
  width: 60%;
  /* height: 60%; */ /* doesn't work */
  padding-bottom: 60%; /* sets height - padding is defined on the width of an element. 60% of parent element's width  */
  background-color: #fdf2e9;
  z-index: -2;
}

.step-img-box::after {
  width: 45%;
  padding-bottom: 45%;
  background-color: #fae5d3;
  z-index: -1;
}
```

Building the Featured-In Section

1. Logs + text

Building the Meals Section - Part 1

1. Problem with SVG icons
    1. Clutters up HTML
        1. Makes it harder to understand
    2. Solution:
        1. ionicons - good for different design personalities
            1. Copy script:

```
<script src="https://unpkg.com/ionicons@5.5.1/dist/ionicons.js></script> <!-- Automatically injects SVG into the page -->
```

            2. Basic usage:

```
<ion-icon name="flame-outline"></ion-icon>
```

Building the Meals Section - Part 2

1. Tags:
    1. Borders are round
2. If images are in cards, they must have the same aspect ratio

Building the Meals Section - Part 3

1. Styling a link:

```
.link:link,
.link:visited {
  display: inline-block;
  color: #e67e22;
  text-decoration: none;
  border-bottom: 1px solid currentColor; /* **(M)**. currentColor - current text color (helps if we change using hover) */
  padding-bottom: 2px;
  transition: all 0.3s;
}

.link:hover,
.link:active {
  color: #cf711f;
  /* border-bottom: none; */ /* messes up the spacing */
  border: 1px solid transparent; /* **(M)** */
}
```

2. Generic class for centering text:

```
.center-text {
    text-align: center;
}
```

Building the Testimonials Section - Part 1

1. 2x2 grid on the left side
    1. `<figure>` - **(M)**
        1. Used to represent some self contained content
        2. Good for diagrams, photos, code-listings

2. Photos on the right side
3. `<blockquote>` - **(M)**
    1. To quote someone

Building the Testimonials Section - Part 2

1. Columns usually need more spacing than rows.

2. Gallery styling:

```
.gallery {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 0.8rem;
  padding: 0.8rem;
}

.gallery-item img {
  display: block; /* to fix spaces for inline elements */
  width: 100%;
  transition: all 0.4s;
}

.gallery-item {
  overflow: hidden;
}

.gallery img:hover {
  /* transform: rotate(45deg); */
  transform: scale(1.1);
}
```

Building the Pricing Section - Part 1

1. Pricing cards & features

Building the Pricing Section - Part 2

2. Styling:

```
.pricing-plan--complete {
  background-color: #fae5d3;
  padding: 2.4rem;
  overflow: hidden;
  position: relative;
}

.pricing-plan--complete::after {
  content: "Best value";
  position: absolute;
  top: 6%;
  right: -18%;
  text-transform: uppercase;
  font-size: 0.7rem;
  font-weight: 700;
  color: #333;
  background-color: #ffd43b;
  padding: 0.4rem 4rem;
  transform: rotate(45deg);
}
```

3. `:not` pseudoclass:

```
.grid:not(:last-child) {
    margin-bottom: 4.8rem;
}
```

Building the Features Part

1. Feature styling:

```
.feature-icon {
  color: #e67e22;
  height: 1.6rem;
  width: 1.6rem;
  background-color: #fdf2e9;
  margin-bottom: 1.6rem;
  padding: 0.8rem;
  border-radius: 50%;
}

.feature-title {
  font-size: 1.2rem;
  color: #333;
  font-weight: 700;
  margin-bottom: 0.8rem;
}

.feature-text {
  font-size: 0.9rem;
  line-height: 1.8;
}
```

Building the Call-To-Action Section - Part 1

1. Left side: sign-up form
2. Right side: supporting image

Building the Call-To-Action Section - Part 2

1. Form elements are inline elements
2. If we click on label, cursor will point to the input field it is associated with
3. If we hit keyboard, a blue border is shown on elements
    1. This can be removed
4. Checkboxes are hard to style - not covered in the course

Building the Call-To-Action Section - Part 3

Building the Footer - Part 1

1. Design:
    1. First column:
        1. Logo
        2. Social links
        3. Copyright
    2. Second column:
        1. Contact info
    3. Remaining columns
        1. Additional links
2. Implementation:
    1. Header and footer might get repeated in other pages
    2. Main part is specific to a page

Building the Footer - Part 2

1. Styling
2. Responsive:
    1. Making it adapt to smaller screens

Section 8: Omnifood Project - Responsive Web Design

Section Intro

1. Using media queries

How Media Queries Work

1. How media queries work (with `max-width`) property (for desktop first strategy) (for mobile-first approach we use `min-width` property)
    1. Example: Applying style between 0 - 600px

        ```
        @media (max-width: 600px)
        ```

        1. Is `width` <= `600px`
            1. If this condition is true, the css inside the media query will apply
            2. If not, it will not apply
    2. They are tools for overriding specific parts of css at certain viewport widths
    3. Consider the situation:

        ```
        @media (max-width: 600px)

        @media (max-width: 1200px)
        ```

        1. Screen size: 400px:
            1. Both will apply
            2. If there is conflict, the one that appears last in the code will apply
2. Responsive version implementation:
    1. Open Dev Tools
        1. Click the devices icon
            1. We can select different devices
                1. Select **Responsive**
                2. Select scale factor to 75%
3. Examples:

```
@media (max-width: 600px) {
  .section-hero {
    background-color: orangered;
  }
}

@media (max-width: 300px) {
  .section-hero {
    border: 20px dashed blue;
  }
}
```

4. How do we know which values to choose and how many media queries do we need to write?

How to Select Breakpoints

Strategies for Selecting Breakpoints

1. Breakpoints: Viewport widths at which want the design to change
    1. Pixel values we want to use for media queries
2. **Bad Strategy**:
    1. Using popular devices such as iPhone or iPad screen sizes
    2. Why?
        1. If we optimize for one device, we ignore the other devices
            1. Doesn't give a good experience for those users
        2. Not maintainable because we need to update the design if the device screen sizes change due to newer versions of the devices or newer devices released by the company (iPhone say)
3. **Good Strategy**
    1. Based on screen width ranges
        1. We look at most used widths for different categories of devices (phones, tablets, desktop computers, ...) and we group them in a logical way to group the breakpoints
        2. Why?
            1. We are using many devices
            2. We are not setting breakpoints for one particular device but for similar device sizes
                1. 600px, 900px, 1200px
                    1. Most phones: 300px to 600px (at 300px)
                    2. Most tablets: 600px to 900px
                    3. Most tablets (landscape): 900px to 1100px
                    4. Most computers: > 1200px (at 1200px)
4. **Perfect Strategy**
    1. Setting breakpoints where design breaks down
        1. Ignore devices and only look at content and design
            1. How to check?
                1. We start at one screen size (largest or smallest)
                2. We increase (for smallest) or decrease (for largest) the screen size
                    1. When we find that the design breaks at a certain point, that is a new breakpoint
        2. It is hard to do this completely without thinking about devices
5. How to go about?
    1. A combination of **Good strategy** and **Perfect strategy**

Responding to Small Laptops

1. HTML must have the following line for responsive design:

    ```
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    ```

    1. Without this line, responsive design does not work on physical devices
        1. Browsers on mobile devices zooms the page out by default, until it fits the screen
            1. We don't want that. We want to make the layout smaller (less wide)
        2. The page matches the device's width
            1. `width=device-width` - page will match the screen's width
            2. `initial-scale=1.0` - 100% (not zoom out of the page)

2. Checking where design breaks

    1. Make it 50%
    2. Start from 1500px
    3. Start making it smaller
        1. It breaks at 1300 (1360px - HD ready screen size)
            1. We can make hero as wide as rest of the page (1200px)
                1. **Don't use pixeles in media queries**
                    1. Does not adjust to the font-size in the browser
                    2. Does not adjust to the zoom-level
                    3. However, they do not respond to the `font-size` setting in `style.css`
                        1. `1rem` will always be the default `font-size` of the browser
        2. **It is not a good practice to add too many media queries**

Responding to Landscape Tablets

1. Continue decreasing the viewport
    1. Text moves to multiple lines
    2. Too much spacing
    3. Too big text
2. `1200px` - mobile device territory
3. Then test in iPad in landscape view

Responding to Tablets

1. Tablets in landscape mode
2. **A media query should work for 200px - 300px**
3. Grid makes it easy to design responsive layouts

Building the Mobile Navigation

1. Nav must disapper and a menu item should appear on the top-right corner

2. We can select element based on attributes:

    ```
    .icon-mobile-nave[name="close-outline"] {
        display: none;
    }
    ```

3. Mechanism to show and hide navigation

    1. Add a class to the element that we want to be visible

4. Code:

```
/* MOBILE NAVIGATION */
.btn-mobile-nav {
  display: block;
}

.main-nav {
  background-color: rgba(255, 255, 255, 0.97);
  position: absolute; /* to the viewport */
  top: 0;
  left: 0;
  width: 100%;
  height: 100vh;
  transform: translateX(100%);

  display: flex;
  align-items: center;
  justify-content: center;
  transition: all 0.5s ease-in;

  /* Hide navigation */
  /* display: none; */ /* Doesn't allow animations - transitions don't work */

  /* 1) Hide it visually */
  opacity: 0; /* not enough. (it exists but becomes invisible) It can be animated, though */

  /* 2) Make it unaccessible to mouse and keyboard (tab keys will go to the links in the hidden element) */
  pointer-events: none; /* **(M)** */

  /* 3) Hide it from screen readers */
  visibility: hidden; /* **(M)** */
}

.nav-open .main-nav {
  /* applied only if this class is used */
  opacity: 1;
  pointer-events: auto; /* **(M)** */
  visibility: visible; /* **(M)** */
  transform: translateX(
    0
  ); /* left could be used. It is not as smooth and not efficient. animation is optimized for `tranform` and `opacity`, etc. */
}

.nav-open .icon-mobile-nav[name="close-outline"] {
  display: block;
}

.nav-open .icon-mobile-nav[name="menu-outline"] {
  display: none;
}

.main-nav-list {
  flex-direction: column;
  gap: 4.8rem;
}

.main-nav-link:link,
.main-nav-link:visited {
  font-size: 3rem;
}
```

Responding to Smaller Tablets

1. Asymmetric grid:

```
.grid-footer {
  grid-template-columns: repeat(6, 1fr);
}

.logo-col,
.address-col {
  grid-column: span 3;
}

.nav-col {
  grid-row: 1;
  grid-column: span 2;
  margin-bottom: 3.2rem;
}
```

Responding to Phones

1. Show media queries:
    1. Click on dots on top-right menu
        1. Show media queries
            1. We can click on the media queries
2. On mobile phones, the buttons have to be big enough to easily tap them
3. **If**
   doesn't have any content, CSS doesn't give any height (unless there is another element beside it)

Section 9: Omnifood Project - Effects, Optimizations and Deployment

Section Intro

1. Topics:
    1. JS to make scrolling and mobile navigation work
    2. Compressing images for better performance

3. Launch project to the internet

A Short Introduction to JavaScript

1. Dynamic effects
    1. Mobile navigation
    2. Scrolling animation
2. Introduction to JS:
    1. Programming language of the web
3. Project:
    1. 08-Omnifood-Optimizations
        1. js/script.js
    2. JS can be written in HTML or in external file
        1. Linking:

```
<script defer src="js/script.js"></script>
```

        2. JS code:

```
console.log("Alhamdulillah!");

const myName = "Jonas Schmedtmann";
const h1 = document.querySelector(".heading-primary");
console.log(h1);

h1.addEventListener("click", function () {
  h1.textContent = myName;
  h1.style.backgroundColor = "red";
  h1.style.padding = "5rem";
});
```

4. Automatically changing date:

```
const yearEl = document.querySelector(".year");
const currentYear = new Date().getFullYear();
yearEl.textContent = currentYear;
```

Making the Mobile Navigation Work

1. Navigating when links a clicked
    1. Page anchor: `#<id>` **(M)**
2. The following will not work with Safari (upto 2024 or 2025 say) and iPhone and iPad phones (Google chrome uses Safari engine in Apple products)

```
scroll-behavior: smooth;
```

    1. Method 2:

```
////////////////////////////////////////////////////////
// Smooth scrolling animation

const allLinks =
  document.querySelectorAll(
    "a:link"
  ); /* Anchor elements that have the href property */
allLinks.forEach(function (link) {
  link.addEventListener("click", function (event) {
    // console.log(event);
    event.preventDefault();
    const href = link.getAttribute("href");
    // console.log(href);

    // Scroll back to top
    if (href === "#") {
      window.scrollTo({
        top: 0 /* 0 px */,
        behavior: "smooth",
      });
    }
  });
});
```

    2. Method 3: Polyfill for smooth scrolling

```
<script defer src="https://unpkg.com/smoothscroll-polyfill@0.4.4/dist/smoothscroll.min.js"></script>
```

Implementing Smooth Scrolling

```
////////////////////////////////////////////////////////
// Smooth scrolling animation

const allLinks =
  document.querySelectorAll(
    "a:link"
  ); /* Anchor elements that have the href property */
allLinks.forEach(function (link) {
  link.addEventListener("click", function (event) {
    // console.log(event);
    event.preventDefault();
    const href = link.getAttribute("href");
```

```
    // console.log(href);

    // Scroll back to top
    if (href === "#") {
      window.scrollTo({
        top: 0 /* 0 px */,
        behavior: "smooth",
      });
    }

    // Scroll to other links
    if (href !== "#" && href.startsWith("#")) {
      // console.log(href);
      const sectionEl = document.querySelector(href); /* href is the id */
      sectionEl.scrollIntoView({
        behavior: "smooth",
      });
    }

    // Close mobile navigation
    if (link.classList.contains("main-nav-link")) {
      headerEl.classList.toggle("nav-open");
    }
  });
});
```

Implementing a Sticky Navigation Bar

```
////////////////////////////////////////////////////////
// Sticky navigation

const sectionHeroEl = document.querySelector(".section-hero");
const obs = new IntersectionObserver(
  function (entries) {
    const ent = entries[0]; // one entry for each threshold value (we have only one in this case)
    console.log(ent);
    if (!ent.isIntersecting) {
      document.body.classList.add("sticky");
    }

    if (ent.isIntersecting) {
      document.body.classList.remove("sticky");
    }
  },
  {
    // In the viewport (inside browser window)
    root: null, // `root` - where the element should be appearing or not,
    threshold: 0, // an event is triggered if 0% of the hero section is inside of the viewport
    rootMargin:
      "-80px" /* It must be pixels. rem or % doesn't work. That is why `height` is set manually */,
  }
); // modern and efficient way
obs.observe(sectionHeroEl); // observe the eleemnt
```

Browser Support and Fixing Flexbox Gap in Safari

1. The processess behind building a website:
    1. **Test and Optimize**
        1. Make sure website works well in **all major browsers** (Chrome, Firefox, Safari, Edge, maybe even old IE)
            1. Important before. Not a big issue now
        2. Test the website on **actual mobile devices**, not just in DevTools
        3. Optimize all **images**, in terms of dimensions and file size (See lecture on images)
        4. Fix simple **accessibility** problems (e.g. color contrast issues)
        5. Run the **Lighthouse** performance test in Chrome DevTools and try to fix reported issues
        6. Think about **Search Engine Optimization (SEO)**

Browser Support

1. There were many inconsistencies among browsers and website would have looked different before
    1. Some browsers did not implement certain properties used at the time.
        1. Those are fixed for most CSS properties
            1. Assuming users are using the updated versions
                1. If there are users using old browsers (IE, Safari etc.), some things might not work
                    1. Solution: Search: caniuse.com
                        1. Enter css property
                            1. `css grid`
                                1. All modern browsers support it
                                    1. Older browsers (especially IEs) don't support all properties
                            2. `backdrop filter` - not supported by Firefox
                                1. Chrome: `-webkit-` prefix must be used

                                ```
                                .main-nav {
                                    backdrop-filter: blur(10px); /* blurs everything behind an element */
                                }
                                ```

                            3. `flex` doesn't work in older Safari browser
2. Writing same media query twice is not a problem
    1. Both of them will apply

Testing Performance With Lighthouse

1. Lighthouse tool:
    1. Automation tool to improve quality of our pages
    2. Developed by Google
        1. It is available in Dev tools
            1. Inspect > Lighthouse
                1. Turn on all Categories (except Progrssive Web App)
                2. Desktop Device
                3. Generate report
                    1. Scores:
                        1. Performance - 100%
                            1. Relevant if the website is deployed
                        2. Accessibility - 98%
                        3. Best Practces: 100%
                        4. SEO - 90%
                    2. Opportunity:
                        1. Properly size images - 1.4 s can be saved
                            1. It is very important to optimize images
                                1. Image dimensions
                                2. Image size
                        2. Payloads - file size
                        3. Explicitly define width and height (impacts performance otherwise)
                    3. Accessibility
                        1. Background and foreground colors do not have a sufficient contrast ratio
                    4. SEO
                        1. Document does not have a meta description
                            1. Short description that will be visible in Google and other search engines

Adding Favicon and Meta Description

1. SEO:
    1. `<meta>` - data that describes other data
2. favicon
    1. We need multiple versions for different devices
        1. favicon for browsers
        2. On iOS
        3. On Android

Favicon

1. Open favicon.png
    1. Resize image
        1. Or take final images from final course files
    2. Defining the favicon

```
<link rel="icon" href="img/favicon_64_by_64.png">
```

2. **There are ways of adding a website as a favorite to iOS and Android devices.**

```
<link rel="apple-touch-icon" href="img/favicon_180_by_180_for_iOS.png">
```

    1. For Android:
        1. Step 1: Add `manifest.webmanifest` in home folder
        2. Step 2: Add the following config:

```
{
  "icons": [
    {
      "src": "img/favicon_192_by_192_for_Android.png",
      "type": "image/png",
      "sizes": "192x192"
    },
    {
      "src": "img/favicon_512_by_512_for_Android.png",
      "type": "image/png",
      "sizes": "512x512"
    }
  ]
}
```

        3. Step 3: Add the following link in `index.html`

```
<link rel="manifest" href="manifest.webmanifest">
```

Image Optimizations

1. In terms of image dimensions and file size

Optimizing Image Dimensions

1. Choose the widest screen:

2. Check the the size of the image
    1. Note the size
3. Double the size of the image (required for HD screens - HD screens use 2 px of the image to display 1 px)
    1. That is the final size (for the screen)

Optimizing Image File Size

1. Checking the size of the page:
    1. Inspect > Network

        1. Refresh the page: 3.9MB transferred (too much!)

            1. Solution: https://squoosh.app
                1. Left side: Original
                2. Right size: Compressed
                    1. JPEG - Doesn't work for transparent parts (transparent parts become black)
                        1. OxiPNG
                            1. Reduce palette - quality goes down
                3. Solution: Webp (reduces a lot) (reduce it to 200kb)
                    1. Increase quality - 92%
                    2. Lossless
                    3. Sign loss: 0
                    4. Effor: Increase
                    5. Reduce palette: off
                    6. Check if browsers support webp

                        1. Safari has less support

                        2. Latest iPhone Safari supports

                        3. ...

                        4. Solution: For multiple browsers:

```
<picture>
    <source srcset="img/hero_1200_compressed.webp" type="image/webp">
    <source srcset="img/hero_1200_compressed.png" type="image/png">

    <img src="img/hero_1200_compressed.webp"
```

alt="Woman enjoying food, meals in storage container, and food bowls on a table" class="hero-img">

1. Browser chooses the source of the image from the given options
2. There is a fallback in case the `<source>` attribute doesn't work

Deployment to Netlify

1. There are many web-hosting providers
    1. Free service: Netlify https://netlify.com
        1. Sign-up
        2. Sites link
        3. Drag and drop a site folder
            1. Git - Better option
            2. Gives a URL
                1. Live version of the website living on the internet
                    1. Site settings:
                        1. Change site name: omnifood-jonas
                    2. https://omnifood-poc.netlify.app/ - Comes with a certificate that is refreshed every 2 months automatically
                        1. We need to purchase a URL for `omnifood.com`
                            1. Godaddy
                            2. hover.com
                    3. Run lighthouse test

Making the Form Work With Netlify Forms

1. Sign-up form
    1. Open site settings
        1. Forms - we can manage forms and submissions without any server-side code or JS
            1. Netlify can handle it:

                1. Add `name="contact"` and `netlify` as follows:

```
<form name="sign-up" netlify>
```

                2. Add `name` attribute to each input field

                    1. Standard attribute for form inputs
                        1. Used to identify the input fields on the server side

                3. Don't use netlify forms for sign-ups or logins with passwords

                    1. There is a separate feature in netlify for that (paid service)

                4. Go to Deploys

                    1. Drag and drop the folder again
                        1. Netlify adds `method="post"`

                5. Go to Forms

        1. Enable form detection
        2. Redeploy the project
        3. We can download the details as csv
            1. Only 100 submissions are allowed per month in the free plan
    2. If there are multiple forms on the page, we need to give a different name

Section 10: The End!

Where to Go from Here

1. Next steps:
    1. Practice, practice, practice!
        1. To become a full developer
            1. How to do that?
                1. Re-create other websites and layouts we like
                    1. Start from simple examples
                    2. Move onto more complex interfaces
                        1. Twitter - say
    2. Never stop learning
        1. A huge field - HTML & CSS are just a beginning
        2. Recommendations:
            1. **JavaScript**
            2. **React** or **Node**
            3. **Advanced CSS**
                1. SASS
            4. **WorkPress or Jamstack** (static website generators - 11ty, jekyll, Next.js (related to React))
        3. Newsletter: jonas.io

My Other Course + Updates

1. JavaScript - the language of the web
2. Advanced CSS (to master flexbox and CSS grid)
    1. Advanced responsive design
    2. Advanced animations
    3. ...
3. Follow:
    1. @jonasschmedtman
    2. Sign up for mailing list
    3. YouTube channel